UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

Testarea Sistemelor Software

Proiect

TESTAREA UNITARĂ ÎN JAVA

Studiu de caz pe clasa WalletServiceImpl din Călătorii

Echipa 13

Iftime Raluca	șef echipă
Lupeș Ioan-Marian	membru
Denisa Gheorghe	membru

Coordonator
Lect. dr. Preduț Sorina Nicoleta

București, aprilie 2026

Rezumat

Lucrarea de față documentează aplicarea unui set de strategii consacrate de testare unitară pe o clasă reală dintr-o aplicație web Spring Boot. Clasa aleasă, WalletServiceImpl, gestionează portofelul fiecărei călătorii din aplicația Călătorii: setarea unui buget, adăugarea și ștergerea cheltuielilor, calculul unor sumarizări KPI și generarea de recomandări inteligente bazate pe ritmul de consum.

Asupra acestei clase au fost aplicate, în mod sistematic, șase strategii de testare: partiționare în clase de echivalență, analiza valorilor de frontieră, acoperire la nivel de instrucțiune, decizie, condiție și acoperire pe circuite independente (basis path). În plus, calitatea suitei a fost evaluată prin Mutation Testing folosind PITest, instrument care generează variante mutate ale codului sursă pentru a verifica dacă testele detectează modificările.

Au fost scrise în total în jur de 40 de teste, organizate pe șapte fișiere distincte (câte unul pentru fiecare strategie), care au atins 91% statement coverage și 80% branch coverage măsurate cu JaCoCo, respectiv un mutation score de 60% pe PITest. Am identificat și omorât în mod explicit doi mutanți neechivalenți (unul de tip MathMutator, unul de tip ConditionalsBoundaryMutator). Suplimentar, am realizat un experiment în care un model AI (ChatGPT) a fost rugat să genereze automat o suită echivalentă pentru aceeași clasă, fără context. Comparația cantitativă și calitativă a celor două suite e prezentată în finalul lucrării.

Cuvinte cheie: testare software, testare unitară, JUnit 5, Mockito, JaCoCo, PITest, mutation testing, Equivalence Partitioning, Boundary Value Analysis, Spring Boot.

Abstract

This work documents the systematic application of a set of established unit testing strategies on a real class from a Spring Boot web application. The chosen class, WalletServiceImpl, manages the wallet of each trip in the Călătorii application: setting a budget, adding and deleting expenses, computing KPI summaries and generating smart recommendations based on consumption rate.

We applied six testing strategies on this class in a structured manner: equivalence partitioning, boundary value analysis, statement coverage, decision coverage, condition coverage and basis path (independent circuits) coverage. Additionally, the quality of the test suite was evaluated through Mutation Testing using PITest, a tool which generates mutated variants of the source code to verify whether the tests detect the changes.

A total of around 40 tests were written, organized in seven separate files (one for each strategy), reaching 91% statement coverage and 80% branch coverage as measured by JaCoCo, and a mutation score of 60% on PITest. We explicitly identified and killed two non-equivalent mutants (one of type MathMutator, one of type ConditionalsBoundaryMutator). As an additional experiment, we asked an AI model (ChatGPT) to automatically generate an equivalent test suite for the same class, without providing context. A quantitative and qualitative comparison of the two suites is presented at the end of this work.

Keywords: software testing, unit testing, JUnit 5, Mockito, JaCoCo, PITest, mutation testing, Equivalence Partitioning, Boundary Value Analysis, Spring Boot.

Cuprins

1 Introducere 11
- 1.1 Context și motivație . . . 11
- 1.2 Obiectivele proiectului . . . 11
- 1.3 Echipa și diviziunea muncii . . . 12
- 1.4 Structura lucrării . . . 12

2 Preliminarii teoretice 14
- 2.1 Testare unitară: ce este și ce nu este . . . 14
- 2.2 Testarea funcțională (Black Box) vs. testarea structurală (White Box) . . . 15
  - 2.2.1 Black Box . . . 15
  - 2.2.2 White Box . . . 15
  - 2.2.3 Mutation Testing . . . 15
- 2.3 Statement coverage . . . 17
- 2.4 Decision coverage . . . 17
- 2.5 Condition coverage . . . 17
  - 2.5.1 Short-circuit-ul Java . . . 18
- 2.6 Basis Path Coverage (Circuits) . . . 18
- 2.7 Mutation Testing - operatori și interpretare . . . 19
  - 2.7.1 Mutanți echivalenți . . . 19
- 2.8 JaCoCo - cum măsoară coverage-ul . . . 20
- 2.9 De ce JUnit 5 + Mockito + JaCoCo + PITest . . . 20

3 Configurația de lucru 22
- 3.1 Hardware . . . 22
- 3.2 Sistem de operare și mașini virtuale . . . 22
- 3.3 Stiva tehnologică . . . 23
  - 3.3.1 Justificarea alegerii instrumentelor de testare . . . 23
- 3.4 Arhitectura proiectului . . . 24
- 3.5 Configurarea Maven (pom.xml) . . . 24
- 3.6 Versiunea de Java și note de compatibilitate . . . 27

4 Aplicația și clasa testată 28
4.1 Aplicația Călătorii . . . 28
4.2 Modulul Wallet - context și roluri . . . 28
4.3 Diagrama claselor . . . 29
4.4 Clasa WalletServiceImpl - prezentare . . . 30
4.4.1 Metodele publice principale . . . 30
4.4.2 Logica de ownership check . . . 30
4.4.3 Pragurile dinamice și enum-ul de stare . . . 31
4.4.4 Buget inteligent (smart advice) - extensie . . . 31
4.5 De ce am ales WalletServiceImpl pentru testare . . . 32

5 Strategiile de testare aplicate 34
5.1 Organizarea suitei . . . 34
5.1.1 Decizia de a aplica strategiile pe metode diferite . . . 34
5.2 Echipamentul comun de test (test fixture) . . . 35
5.3 Equivalence Partitioning . . . 37
5.3.1 Metoda țintă: addExpenseOwnedByUser . . . 37
5.3.2 Exemplu de test EP . . . 37
5.3.3 Numărul de teste pentru EP . . . 38
5.4 Boundary Value Analysis . . . 38
5.4.1 Frontiere pentru addExpenseOwnedByUser . . . 38
5.4.2 Frontiere pentru cascada de praguri . . . 38
5.4.3 Numărul de teste pentru BVA . . . 39
5.5 Statement Coverage . . . 39
5.5.1 Metoda țintă: updateBudgetOwnedByUser . . . 39
5.5.2 Numărul de teste pentru Statement . . . 40
5.6 Decision Coverage . . . 40
5.6.1 Metoda țintă: computeSmartBudgetAdviceOwnedByUser . . . 40
5.6.2 CFG pentru buildRiskUi . . . 41
5.6.3 Cele 5 ramuri testate . . . 41
5.7 Condition Coverage . . . 42
5.7.1 Cele 6 condiții compuse țintite . . . 42
5.7.2 Exemplu detaliat: C3 cu trei operanzi OR . . . 42
5.7.3 Numărul de teste pentru Condition . . . 43
5.8 Basis Path / Circuits Coverage . . . 43
5.8.1 Metode țintite . . . 43
5.8.2 Exemplu detaliat: computeDaysRemainingSafe . . . 43
5.8.3 Numărul de teste pentru Circuits . . . 45
5.9 Concluzii capitol . . . 45

6 Rezultate JaCoCo 46
6.1 Pipeline-ul de testare 46
6.2 Cum rulăm JaCoCo 46
6.3 Rezultate generale 47
6.4 Rezultate per metodă 47
6.5 Interpretarea metodelor cu coverage incomplet 48
6.5.1 computeInsightsOwnedByUser (50% branch) 48
6.5.2 findUserByUsernameOrEmail (62% branch) 49
6.5.3 buildRiskUi și buildRecommendation (76%, 91% branch) 49
6.6 De ce nu am forțat 100% 49
6.7 Comparație: Statement vs. Branch Coverage 50
6.8 Concluzii capitol 50

7 Mutation Testing cu PITest 51
7.1 Cum rulăm PITest 51
7.2 Rezultate generale 51
7.3 Defalcare pe tipuri de mutatori 52
7.3.1 Puncte forte ale suitei 52
7.3.2 Puncte slabe ale suitei 53
7.4 Analiza unor mutanți supraviețuitori 54
7.4.1 Mutant supraviețuitor 1: ConditionalsBoundary pe linia 191 54
7.4.2 Mutant supraviețuitor 2: MathMutator pe linia 187 54
7.5 Mutanții neechivalenți omorâți explicit 54
7.5.1 Mutantul 1: MathMutator pe multiply / divide 55
7.5.2 Mutantul 2: ConditionalsBoundary pe percent >= 75 55
7.6 Cum identificăm mutanți echivalenți 56
7.6.1 Exemplu 1: ++i vs. i++ într-un for 56
7.6.2 Exemplu 2: schimbarea ordinii de evaluare la a || b fără side-effects 56
7.7 Concluzii capitol 56

8 Suita generată de AI - experiment și comparație 58
8.1 Scopul experimentului 58
8.2 Promptul folosit 58
8.3 Răspunsul generat 59
8.4 Rezultatul rulării: Build Failed 60
8.4.1 47 de erori de compilare 60
8.4.2 Categoria 1: Clase inventate, inexistente în proiect 61
8.4.3 Categoria 2: Metode inventate pe modele 61
8.4.4 Categoria 3: Metode inventate pe repository-uri 62
8.4.5 Categoria 4: Tipuri de date incompatibile 62

8.4.6 Categoria 5: Import greșit critic . . . 62
8.5 Comparație cantitativă suita proprie vs. suita AI . . . 63
8.6 Analiza diferențelor . . . 64
8.6.1 Unde AI-ul ajută . . . 64
8.6.2 Unde AI-ul greșește . . . 64
8.6.3 O observație colaterală . . . 65
8.7 Concluzia experimentului . . . 65

9 Concluzii 66
9.1 Recapitulare obiective vs. realizat . . . 66
9.2 Ce am învățat . . . 66
9.2.1 Despre proiect . . . 66
9.2.2 Despre folosirea AI-ului . . . 67
9.3 Direcții viitoare . . . 67
9.3.1 Creșterea Mutation Score peste 75% . . . 68
9.3.2 Property-Based Testing . . . 68
9.3.3 Integrare în CI/CD . . . 68
9.3.4 Testare de integrare cu Testcontainers . . . 68
9.4 Considerații finale . . . 68

Bibliografie 71

A Cod sursă relevant 72
A.1 Fragmente din WalletServiceImpl . . . 72
A.1.1 Constructorul cu injecție de dependențe . . . 72
A.1.2 Metoda addExpenseOwnedByUser . . . 73
A.1.3 Metoda computeDaysRemainingSafe . . . 73
A.2 Fragmente din suita de teste . . . 74
A.2.1 Configurare comună . . . 74
A.2.2 Test pentru clasa de echivalență P4 (sumă invalidă) . . . 75
A.2.3 Test pentru BVA pe pragul de 75% . . . 75
A.2.4 Test pentru Condition Coverage cu 3 operanzi OR . . . 76
A.2.5 Test țintit pentru mutantul MathMutator . . . 76
A.2.6 Test pentru Circuits pe computeDaysRemainingSafe . . . 77
A.3 CFG pentru addExpenseOwnedByUser . . . 78

Listă de figuri

2.1 Ierarhia strategiilor de testare aplicate în proiect, organizate pe ramurile Black Box / White Box / Mutation, cu numărul de teste scrise pentru fiecare strategie. 16

3.1 Arhitectura aplicației Călătorii pe trei straturi (prezentare, business, persistență), cu evidențierea clasei testate WalletServiceImpl. 25

4.1 Diagrama claselor pentru modulul Wallet, cu accent pe WalletServiceImpl - clasa testată. 29

5.1 Distribuția testelor pe cele șase strategii + Mutation. Culorile evidențiază clasificarea Black Box / White Box / Mutation. 35
5.2 Cascada de praguri în computeSummaryOwnedByUser (75% / 90% / 100%) și valorile testate prin BVA (74, 75, 99, 100). Triunghiurile albastre indică valorile efectiv folosite în teste. 39
5.3 CFG pentru buildRiskUi: cascadă de 4 if-uri care decid label-ul de risc (Buget nesetat / Depășit / Critic / Atenție / OK). 41
5.4 CFG pentru computeDaysRemainingSafe, cu 8 noduri și 12 muchii. V(G) = 6 indică numărul de circuite independente prin graf. 44

6.1 Pipeline-ul de testare: suita JUnit + Mockito alimentează atât JaCoCo (coverage) cât și PITest (mutation testing). 46
6.2 Captură din raportul JaCoCo HTML, vizualizat în browser. 47
6.3 Coverage JaCoCo per metodă, cu prag minim de 80% marcat cu linie roșie. 48

7.1 Captură din raportul PITest HTML, cu sumarul general. 52
7.2 Distribuția mutanților pe tipuri (PITest). Verde = omorâți, roșu = supraviețuitori. 53

8.1 Captură de ecran cu promptul trimis în ChatGPT. 59
8.2 Răspunsul ChatGPT - prima parte a codului generat. 59
8.3 Răspunsul ChatGPT - partea a doua a codului generat. 60
8.4 Răspunsul ChatGPT - partea finală a codului generat. 60

8.5 Captură din IntelliJ IDEA cu erorile de compilare ale suitei generate de AI. 61
8.6 Comparația vizuală suita proprie vs. suita AI pe metricile principale. . . . 63

A.1 CFG pentru addExpenseOwnedByUser: trei decizii principale (sumă, titlu, dayIndex) cu trei branch-uri de eroare și o cale de succes. 79

Listă de tabele

3.1 Configurația hardware folosită pentru dezvoltare și testare. 22
3.2 Stiva tehnologică a aplicației și a suitei de teste. 23

4.1 Metodele publice din WalletServiceImpl, ordonate după complexitate. . . . 30
4.2 Compararea claselor candidat pentru testare unitară. 32

5.1 Organizarea suitei de teste, pe strategii. 34
5.2 Strategia aplicată și metoda principală țintită. 36
5.3 Clase de echivalență pentru addExpenseOwnedByUser. 37
5.4 Valorile de frontieră testate pentru addExpenseOwnedByUser. 38
5.5 Valorile testate pentru cascada de praguri. 38
5.6 Ramurile din buildRiskUi și valorile de input care le activează. 42
5.7 Condițiile compuse din WalletServiceImpl și operatorii. 42
5.8 Condition Coverage pentru C3 (3 operanzi OR cu short-circuit). 42
5.9 Metodele țintite pentru Basis Path Coverage. 43
5.10 Circuitele independente pentru computeDaysRemainingSafe. 45

6.1 Rezultatele JaCoCo pe WalletServiceImpl. 47
6.2 Coverage JaCoCo per metodă (sortat descrescător după branch coverage). 48

7.1 Rezultate generale PITest pe WalletServiceImpl. 52
7.2 Distribuția mutanților pe tipuri de mutatori. 53

8.1 Clase folosite de AI vs. clasele reale din proiect. 61
8.2 Metode inventate de AI pe entitățile JPA. 62
8.3 Metode inventate de AI pe repository-urile JPA. 62
8.4 Comparație detaliată suita proprie vs. suita generată de AI. 63

9.1 Obiectivele proiectului și realizarea lor. 66

Capitolul 1

Introducere

1.1 Context și motivație

Testarea software a devenit una dintre activitățile esențiale în ingineria modernă a aplicațiilor. Costul unui defect descoperit târziu, în producție, poate fi de mai multe ori mai mare decât costul unui defect identificat în timpul dezvoltării [13]. Mai mult, în domeniile sensibile - financiar, medical, sau în servicii care manipulează date personale - bug-urile pot duce la pierderi materiale concrete și la deteriorarea încrederii utilizatorilor. Practic, software-ul fără teste e o promisiune care nu se poate verifica.

În contextul disciplinei Testarea Sistemelor Software, proiectul nostru are ca scop aplicarea practică a strategiilor de testare unitară pe o clasă dintr-o aplicație reală. Nu pe un exemplu didactic simplificat, ci pe cod care funcționează efectiv într-o aplicație web și care prezintă toate caracteristicile incomode ale codului real: dependențe multiple, interacțiuni cu o bază de date, validări complexe, calcule monetare cu BigDecimal, condiții logice compuse cu trei sau patru operanzi.

Aplicația aleasă pentru studiul de caz se numește Călătorii și e un planificator de vacanțe dezvoltat de noi pe stiva Spring Boot 3 + JPA + Thymeleaf. Are funcționalități multiple: gestiune călătorii, jurnal de călătorie, achiziție de tururi, leaderboard, hartă interactivă a țărilor vizitate, chatbot. Modulul pe care l-am ales pentru testare unitară este modulul Wallet, mai precis clasa WalletServiceImpl, care răspunde de logica financiară a fiecărei călătorii.

1.2 Obiectivele proiectului

Cerințele proiectului T3 stabilesc, pentru fiecare echipă, următoarele obiective [6]:

- Alegerea unei singure clase relevante și aplicarea unor teste unitare pe ea, ilustrând cele șase strategii fundamentale: partiționare în clase de echivalență, analiza valorilor de frontieră, acoperire la nivel de instrucțiune, decizie, condiție și pe circuite

independente.

- Generarea unui raport de mutanți cu un instrument dedicat (PITest pentru Java) și omorârea explicită a cel puțin doi mutanți neechivalenți.
- Realizarea unui experiment cu un instrument AI care generează teste automat (ChatGPT, GitHub Copilot etc.), urmat de o comparație detaliată cu suita proprie.
- Documentarea completă a procesului, cu strategii, diagrame, configurația software și hardware, capturi de ecran cu rularea testelor, comparații tabelare și interpretări.

În plus, ne-am setat și câteva obiective interne, care depășesc strict cerințele:

- Să atingem cel puțin 80% pe branch coverage măsurat cu JaCoCo, ca prag minim pentru o suită considerată solidă.
- Să atingem cel puțin 60% mutation score, ținând cont de complexitatea ridicată a clasei alese.
- Să demonstrăm că aplicăm strategiile cu rigoare - nu doar din punct de vedere cantitativ, ci și conceptual, justificând în documentație de ce a fost ales fiecare test în parte.

1.3 Echipa și diviziunea muncii

Proiectul a fost realizat de o echipă de trei studenți. Munca a fost împărțită astfel încât fiecare membru să contribuie la fiecare etapă majoră:

- Iftime Raluca (șef echipă) - dezvoltarea aplicației Călătorii, integrarea modulului wallet, configurarea instrumentelor de testare (Maven, JaCoCo, PITest), redactarea documentației.
- Lupeș Ioan-Marian - dezvoltarea aplicației, integrarea funcționalităților secundare (chatbot, hartă), generarea capturilor de ecran și a videoclipurilor pentru livrabile.
- Denisa Gheorghe - dezvoltarea aplicației, contribuții la modulul wallet, suport pentru integrarea testelor și pentru experimentul cu AI.

1.4 Structura lucrării

Lucrarea este organizată în nouă capitole, urmate de o secțiune de bibliografie și o anexă cu fragmente de cod relevante. Pe scurt:

- Capitolul 2 - prezentarea fundamentelor teoretice ale testării software, cu accent pe strategiile aplicate (Black Box / White Box / Mutation).
- Capitolul 3 - configurația de lucru: hardware, sistem de operare, versiunile instrumentelor folosite.
- Capitolul 4 - descrierea aplicației Călătorii și a clasei WalletServiceImpl, cu motivarea alegerii.
- Capitolul 5 - aplicarea celor șase strategii pe clasa noastră, cu exemple, tabele, diagrame de control flow și fragmente de cod.
- Capitolul 6 - rezultate JaCoCo (statement coverage și branch coverage), capturi de ecran, interpretarea procentelor neacoperite.
- Capitolul 7 - rezultate PITest, analiza mutanților supraviețuitori, mutanții neechivalenți pe care i-am omorât explicit.
- Capitolul 8 - raport asupra folosirii unui tool AI (ChatGPT) pentru generarea automată de teste, cu prompt, răspuns, capturi de ecran și comparație cantitativă cu suita proprie.
- Capitolul 9 - concluzii finale și direcții viitoare.

Anexa A conține fragmente de cod folosite ca referință pe parcursul documentației, iar bibliografia conține toate sursele consultate, inclusiv documentațiile oficiale ale instrumentelor folosite și articolele didactice de la curs.

Capitolul 2

Preliminarii teoretice

În acest capitol prezentăm, pe scurt, fundamentele teoretice ale strategiilor de testare aplicate ulterior. Capitolul nu pretinde să fie exhaustiv: ne concentrăm exact pe cunoștințele pe care le folosim direct în proiect, fără a deschide subiecte tangențiale (de exemplu, testarea de integrare, end-to-end, sau testarea performanței).

2.1 Testare unitară: ce este și ce nu este

Testarea unitară este nivelul cel mai de jos al piramidei de testare [10]. La acest nivel, scopul este verificarea unei singure unități de cod (în Java, de regulă o clasă sau o metodă) izolată de restul sistemului.

Această izolare e critică: dacă ar trebui să rulăm testele cu o bază de date reală, cu un context Spring complet ridicat și cu rețea, nu am mai vorbi despre teste unitare ci despre teste de integrare sau end-to-end. Izolarea aduce două avantaje practice mari:

1. Viteza. Testele unitare rulează în milisecunde, deci pot fi rulate de mii de ori pe zi (la fiecare commit, la fiecare salvare de fișier în IDE).

2. Determinismul. Un test unitar nu depinde de starea unei baze de date externă, de timpul curent al sistemului, sau de un endpoint HTTP. Dacă cade, motivul e în cod, nu în mediu.

Pentru a obține izolarea în Java, instrumentul standard este Mockito, o bibliotecă care înlocuiește dependențele reale (în cazul nostru: cele patru repository-uri JPA) cu obiecte simulate (mock objects) ce returnează exact ce vrem noi în fiecare test [12].

2.2 Testarea funcțională (Black Box) vs. testarea structurală (White Box)

Cele șase strategii pe care le aplicăm în proiect se împart, conform literaturii [13, 3], în două mari familii.

2.2.1 Black Box

În testarea Black Box, testerul nu se uită la codul sursă. Pleacă de la specificație: ce trebuie să facă funcția pentru anumite intrări și ce să returneze. În această familie intră:

- Equivalence Partitioning (EP) - input-urile sunt împărțite în clase echivalente, în care comportamentul așteptat este același pentru orice valoare. Se alege apoi un singur reprezentant pe clasă.
- Boundary Value Analysis (BVA) - se testează valori aflate pe frontierele dintre clase, deoarece bug-urile clasice apar exact la < vs. <=.

2.2.2 White Box

În testarea White Box, testerul are acces la codul sursă și proiectează testele astfel încât să acopere structura reală a codului [5]. Strategiile uzuale se ordonează ca o ierarhie:

- Statement Coverage - fiecare instrucțiune să fie executată cel puțin o dată.
- Decision Coverage - fiecare decizie (if, while, ternary) să fie evaluată atât pe ramura TRUE, cât și pe FALSE.
- Condition Coverage - pentru condiții compuse (&&, ||), fiecare sub-condiție să fie evaluată independent pe TRUE și pe FALSE.
- Basis Path Coverage (Circuits) - parcurgerea tuturor circuitelor independente prin graful de control al metodei.

Figura 2.1 ilustrează ierarhia și relațiile dintre strategii. Cele de pe ramura White Box se extind unele pe altele: dacă ai Condition Coverage, ai automat și Decision Coverage, care la rândul lui implică Statement Coverage.

2.2.3 Mutation Testing

A treia familie - care nu e nici Black Box, nici White Box clasic - este Mutation Testing. Aici nu se testează comportamentul codului, ci se testează calitatea testelor înseși [4, 16].

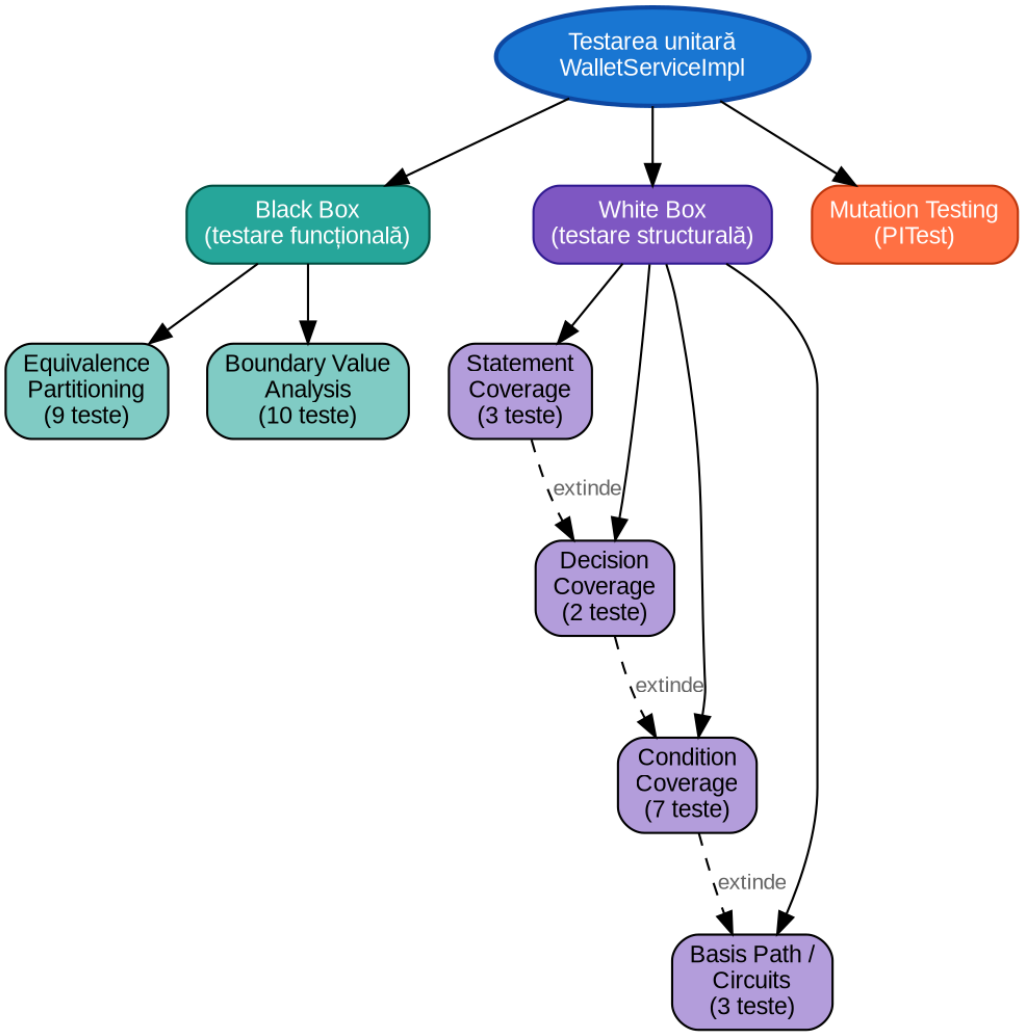


Figura 2.1: Ierarhia strategiilor de testare aplicate în proiect, organizate pe ramurile Black Box / White Box / Mutation, cu numărul de teste scrise pentru fiecare strategie.

Metoda funcționează astfel: instrumentul (în cazul nostru PITest) creează în mod automat copii mutate ale codului sursă - de exemplu, schimbă > cu >=, înlocuiește || cu &&, neagă o condiție - și rulează testele tale pe fiecare copie. Dacă măcar un test cade, mutația e considerată omorâtă: înseamnă că suita a detectat alterarea. Dacă toate testele trec, mutația a supraviețuit: a trecut prin testele tale fără să fie detectată, ceea ce înseamnă o lacună în suită.

Mutation score = procentul de mutanți omorâți din totalul mutanților generați.

Scorul de mutation e o metrică mult mai severă decât statement sau branch coverage, pentru că poate evidenția situații în care testele ating codul, dar nu îl verifică - de exemplu, un test care doar invocă o metodă fără să facă niciun assert pe rezultat.

2.3 Statement coverage

Statement Coverage este criteriul cel mai relaxat dintre cele structurale: cere doar ca fiecare linie de cod (mai exact, fiecare instrucțiune executabilă) să fie executată cel puțin o dată de teste. Nu cere ca toate ramurile if/else să fie acoperite.

Exemplu - dat fragmentul:

```
if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
    throw new IllegalArgumentException("Suma trebuie sa fie > 0");
}
```

Pentru Statement Coverage e suficient un singur test care intră pe ramura adevărată (de exemplu amount = null). Linia throw e executată, deci toate instrucțiunile sunt acoperite. Dar nu am testat ce se întâmplă în cazul amount = 100 - am ratat ramura falsă.

2.4 Decision coverage

Decision Coverage este criteriul branch: pentru fiecare decizie din cod, atât rezultatul TRUE cât și FALSE trebuie să fi fost observate. La exemplul anterior, pentru a satisface Decision Coverage, avem nevoie de cel puțin două teste: unul cu amount = null (intră pe TRUE) și unul cu amount = 100 (intră pe FALSE).

Acesta este criteriul măsurat efectiv de JaCoCo în coloana Branches. Un branch coverage de 100% nu garantează absența bug-urilor, dar e un prag minim profesional rezonabil. Comunitatea Google raportează un nivel de coverage pe proiectele lor între 60% și 70%, în funcție de domeniu [8].

2.5 Condition coverage

Condition Coverage merge un nivel mai jos decât Decision: pentru orice condiție compusă (cu && sau ||), fiecare sub-condiție trebuie evaluată separat pe TRUE și pe FALSE.

Importanța acestei distincții apare la condițiile cu mai mulți operanzi. De exemplu:

```
if (wallet == null || tx.getId() == null
        || !Objects.equals(tx.getWallet().getId(), wallet.getId())) {
    throw new IllegalStateException("Nu ai acces la aceasta tranzactie.")
    ;
}
```

Decision Coverage se mulțumește cu două teste: unul în care expresia totală e adevărată (oricare din cele trei sub-condiții poate fi adevărată) și unul în care e falsă (toate sunt

false). Condition Coverage cere însă ca fiecare din cele trei sub-condiții să fie observată TRUE și FALSE, ceea ce duce la cel puțin 4 teste distincte.

2.5.1 Short-circuit-ul Java

Java evaluează operatorii || și && cu short-circuit: dacă primul operand poate decide rezultatul, al doilea nu mai e evaluat. La ||, dacă primul e TRUE, restul nu se mai evaluează. La &&, dacă primul e FALSE, restul nu se mai evaluează.

În practică, asta înseamnă că pentru o condiție a || b cu doi operanzi, nu putem genera toate cele patru combinații posibile - (T,T), (T,F), (F,T), (F,F) - pentru că în primele două perechi, valoarea lui b nu se evaluează niciodată. Vom avea practic doar trei cazuri distincte:

1. a = TRUE (b nu contează, e short-circuitat)
2. a = FALSE, b = TRUE
3. a = FALSE, b = FALSE

Acest detaliu e important atunci când planificăm testele pentru condiții compuse - nu avem voie să cerem combinații imposibile.

2.6 Basis Path Coverage (Circuits)

Basis Path Coverage, sau acoperirea pe circuite independente, e cel mai sofisticat criteriu structural. Aici nu mai privim deciziile individual, ci la fluxul de control al metodei ca un întreg.

Pentru fiecare metodă, construim Control Flow Graph-ul (CFG): un graf orientat în care nodurile sunt blocuri liniare de cod, iar muchiile sunt transferuri de control (ramuri if, return-uri, throw-uri).

Pentru un graf CFG, definim complexitatea ciclomatică a lui McCabe [11] ca:

$$V(G) = E - N + 2 \tag{2.1}$$

unde E este numărul de muchii și N numărul de noduri. O formulă echivalentă, mai practică:

$$V(G) = \text{numărul de predicate} + 1 \tag{2.2}$$

Complexitatea ciclomatică reprezintă numărul minim de căi independente prin graf, adică numărul minim de teste necesare pentru a parcurge orice combinație de ramuri.

Exemplu. Pentru metoda computeDaysRemainingSafe din clasa noastră, CFG-ul are 8 noduri și 12 muchii (vezi figura 5.4). Aplicând formula:

$$V(G) = 12 - 8 + 2 = 6$$

Deci avem nevoie de cel puțin 6 teste pentru a parcurge toate căile independente. Aceste 6 căi corespund celor 6 scenarii distincte din metodă: start = null, end = null, interval invalid, călătorie încheiată, călătorie viitoare, călătorie în desfășurare.

2.7 Mutation Testing - operatori și interpretare

PITest suportă mai multe seturi de operatori de mutație. Pentru proiectul nostru am folosit setul STRONGER, care e mai cuprinzător decât setul implicit [15]. Cele mai relevante categorii de mutatori sunt:

ConditionalsBoundaryMutator - schimbă operatorii < cu <=, > cu >= și invers. E mutatorul cel mai eficient pentru a forța testele să distingă valorile exacte de pe granițe.

NegateConditionalsMutator - inversează condițiile (de ex. a == b devine a != b). Verifică dacă testele detectează inversarea logică.

MathMutator - schimbă operatorii aritmetici: + cu -, * cu / etc. Util pentru verificarea calculelor matematice.

NullReturnValsMutator - face metodele să returneze null. Verifică dacă testele detectează valori nule pe care nu le-ar fi așteptat.

VoidMethodCallMutator - șterge apelul la o metodă void. Util pentru a verifica dacă side-effect-urile sunt observate de teste (de exemplu, prin verify() pe mock-uri).

RemoveConditionalMutator - elimină complet condiția (transformă if (cond) în if (true) sau if (false)). Are două variante: EQUAL pentru egalități, ORDER pentru comparații.

2.7.1 Mutanți echivalenți

Un caz important pe care literatura îl tratează cu seriozitate este al mutanților echivalenți [14]. Un mutant e echivalent dacă, deși codul a fost modificat sintactic, comportamentul lui rămâne identic cu al codului original pentru orice intrare. Astfel de mutanți nu pot fi omorâți de niciun test, pentru că nu există un test care să producă vreo diferență observabilă.

Exemplu trivial: într-un for care iterează exact o dată, schimbarea limitei superioare cu o valoare mai mare (dar tot < Integer.MAX_VALUE) nu schimbă nimic dacă bucla iese după prima iterație printr-un break. Un astfel de mutant supraviețuiește pentru că, semantic, codul a rămas același.

Distincția între mutanți neechivalenți (care pot fi omorâți și deci indică o lacună reală în suită) și echivalenți (pe care îi documentăm ca atare) e o parte importantă a lucrului cu Mutation Testing. Cerința T3 cere explicit identificarea și omorârea a cel puțin doi mutanți neechivalenți.

2.8 JaCoCo - cum măsoară coverage-ul

JaCoCo [2] e instrumentul standard pentru măsurarea coverage-ului în Java. Funcționează prin instrumentare la nivel de bytecode: înainte de rularea testelor, modifică clasele compilate pentru a injecta sonde (probes) care înregistrează ce instrucțiuni s-au executat.

Avantajul instrumentării bytecode-ului față de instrumentarea la nivelul codului sursă este precizia: JaCoCo măsoară exact ce s-a executat la nivelul JVM, nu o aproximare bazată pe linii de cod. În plus, raportul final indică nu doar instrucțiunile acoperite, ci și ramurile (branches), liniile complexe, metodele atinse și clasele. Pentru proiectul nostru, urmărim în special:

- Instructions Coverage - corespunde aproximativ noțiunii teoretice de Statement Coverage, dar la nivel de bytecode.
- Branches Coverage - corespunde direct Decision Coverage.

2.9 De ce JUnit 5 + Mockito + JaCoCo + PITest

Stiva de instrumente folosită este combinația standard în ecosistemul Java profesional [9]. Toate cele patru instrumente sunt mature, bine documentate și se integrează unele cu altele:

- JUnit 5 (modulul Jupiter) e framework-ul principal de testare. Permite scrierea testelor cu adnotări (@Test, @BeforeEach), sistem de extensii (@ExtendWith) și asserțiuni declarative (assertEquals, assertThrows).
- Mockito se integrează cu JUnit 5 prin MockitoExtension și permite mock-uirea dependențelor cu @Mock și injectarea lor cu @InjectMocks.
- JaCoCo se integrează ca plugin Maven și instrumentează automat clasele înainte de rularea testelor.

- PITest se integrează tot ca plugin Maven și folosește testele JUnit existente pentru a evalua mutațiile.

În capitolele următoare, prezentăm aplicarea efectivă a acestor concepte pe clasa WalletServiceImpl.

Capitolul 3

Configurația de lucru

În acest capitol descriem configurația concretă pe care a fost dezvoltat și testat proiectul: hardware, sistem de operare, alegerea de a nu folosi mașină virtuală, și versiunile exacte ale instrumentelor.

3.1 Hardware

Dezvoltarea s-a desfășurat în principal pe stația de lucru personală a șefei de echipă, cu următoarele specificații:

Tabela 3.1: Configurația hardware folosită pentru dezvoltare și testare.

Componentă	Specificație
Procesor	Intel Core i7-11800H, 8 nuclee, 16 fire, 2.30 GHz
Memorie RAM	16 GB DDR4
Stocare	SSD NVMe 512 GB
Placă video	NVIDIA GeForce RTX 3050 (4 GB)

Cele trei membre ale echipei au folosit configurații similare, toate suficient de puternice pentru rularea suitei de teste și a instrumentelor de coverage. Suita completă (mvn test) rulează în aproximativ 2-3 secunde, iar PITest, care e cel mai costisitor, durează între 5 și 15 minute pentru cei 211 mutanți generați.

3.2 Sistem de operare și mașini virtuale

Sistemul de operare folosit a fost Windows 11 Pro (build 22H2), cu PowerShell ca shell pentru rularea comenzilor Maven. Comenzile sunt portabile și pe Linux/macOS dacă proiectul ar fi clonat acolo - Maven Wrapper (mvnw) e inclus în repository și are atât versiune .cmd pentru Windows, cât și script Unix.

Nu a fost folosită nicio mașină virtuală. Toate instrumentele (JDK, Maven, IntelliJ IDEA, Postgres pentru rularea aplicației) au fost instalate nativ pe Windows. Decizia a fost luată din considerente de performanță: rularea unei suite de teste cu Mockito e ușor afectată de overhead-ul unei VM, iar pentru PITest - care rulează n variante mutate ale codului, fiecare cu suita completă - timpul total ar fi crescut semnificativ.

Pentru baza de date am folosit PostgreSQL 16 pentru aplicație și H2 in-memory pentru cazurile în care a fost nevoie să rulăm aplicația fără backend extern. Atenție însă: testele unitare nu folosesc nici Postgres, nici H2 - dependențele sunt complet mock-uite cu Mockito, deci suita rulează independent de orice bază de date.

3.3 Stiva tehnologică

Aplicația Călătorii e construită pe Java 17 și Spring Boot 3.3.4. Tabelul 3.2 sintetizează componentele majore.

Tabela 3.2: Stiva tehnologică a aplicației și a suitei de teste.

Categorie	Tool / framework	Versiune
Limbaj	Java (OpenJDK)	17
Framework principal	Spring Boot	3.3.4
ORM	Hibernate (prin Spring Data JPA)	6.5.x
Templating	Thymeleaf	3.1.2
Bază de date (prod)	PostgreSQL	16
Bază de date (dev)	H2 in-memory	2.x
Build tool	Maven	3.9.11 (prin wrapper)
Generare cod	Lombok	1.18.42
Test framework	JUnit Jupiter	5.10.3
Mocking	Mockito (mockito-junit-jupiter)	5.11.0
Coverage	JaCoCo (Maven plugin)	0.8.11
Mutation Testing	PITest (pitest-maven)	1.16.3
IDE	IntelliJ IDEA Ultimate	2024.3.5
Sistem de control versiuni	Git	2.45.x
Hosting repo	GitHub	N/A

3.3.1 Justificarea alegerii instrumentelor de testare

Stiva JUnit 5 + Mockito + JaCoCo + PITest este, de facto, standardul pentru testare unitară pe ecosistemul Java în 2026. Câteva motive concrete pentru fiecare alegere:

JUnit 5 (Jupiter) este versiunea modernă, modulară, a JUnit-ului. Față de JUnit 4 oferă: metode de test package-private (nu mai trebuie public), un sistem de extensii

curat prin @ExtendWith, asserțiuni îmbunătățite (assertThrows, assertAll), suport nativ pentru parametrizate. Vine implicit prin spring-boot-starter-test, deci nu trebuie adăugat manual.

Mockito este alegerea standard pentru mock-uri în Java, cu o sintaxă fluentă (when(...).thenReturn(...)) și integrare elegantă cu JUnit 5. Alternativele - EasyMock, PowerMock - sunt fie mai vechi, fie au probleme cu noile versiuni de Java.

JaCoCo este, de asemenea, instrumentul standard în ecosistemul Java, integrat cu Maven, Gradle și majoritatea IDE-urilor. Generează rapoarte HTML interactive în care fiecare linie e colorată în verde (acoperită), galben (parțial acoperită) sau roșu (neatinsă).

PITest este instrumentul lider pentru Mutation Testing pe Java, cu performanță bună și un set bogat de mutatori. Alternativele (MuJava, Major) sunt mai academice, mai puțin întreținute și mai dificil de integrat cu Maven.

3.4 Arhitectura proiectului

Aplicația respectă structura clasică layered a unui proiect Spring Boot, vizibilă în figura 3.1:

- Stratul de prezentare - controllere Spring MVC + template-uri Thymeleaf. Pentru wallet, există un singur WalletController care expune endpoint-urile HTTP.
- Stratul de business - serviciile cu logica aplicativă. WalletServiceImpl e implementarea concretă a interfeței WalletService și e clasa pe care o testăm în acest proiect.
- Stratul de persistență - patru repository-uri JPA pe care WalletServiceImpl le folosește prin injecție de dependențe: TripRepository, TripWalletRepository, UserRepository, WalletTransactionRepository.
- Baza de date - PostgreSQL pentru rularea efectivă a aplicației.

În testele unitare, întreg stratul de persistență e mock-uit, ceea ce face ca testele să fie rapide și deterministe.

3.5 Configurarea Maven (pom.xml)

Plugin-urile principale relevante pentru testare sunt configurate în pom.xml astfel.
Pentru JaCoCo:

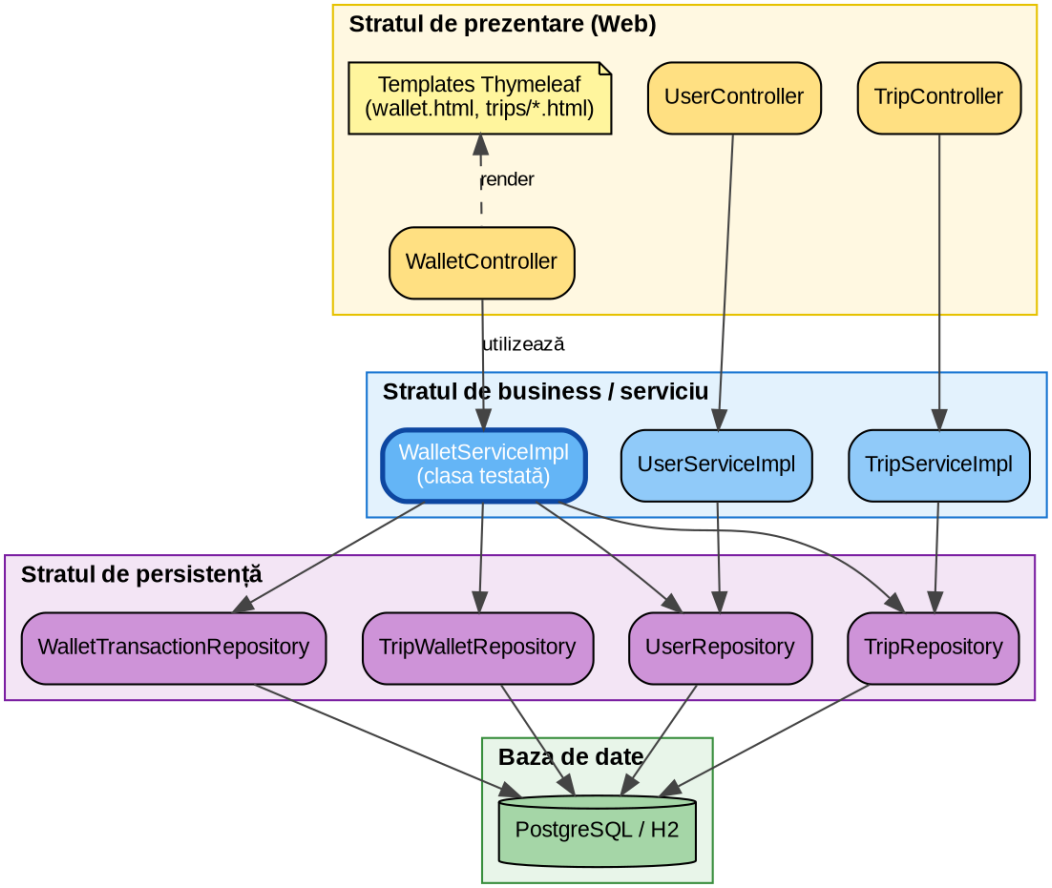


Figura 3.1: Arhitectura aplicației Călătorii pe trei straturi (prezentare, business, persistență), cu evidențierea clasei testate WalletServiceImpl.

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.11</version>
    <executions>
        <execution>
            <id>prepare-agent</id>
            <goals>
                <goal>prepare-agent</goal>
            </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals>
                <goal>report</goal>
            </goals>
            <configuration>
                <includes>
```

```
                    <include>echipa13/calatorii/service/impl/
    WalletServiceImpl.class</include>
                </includes>
            </configuration>
        </execution>
    </executions>
</plugin>
```

Listing 3.1: Configurarea pluginului JaCoCo în pom.xml.

Filtrul includes limitează raportul JaCoCo la o singură clasă - WalletServiceImpl. Decizia e deliberată: cerința proiectului e să testăm o singură clasă reprezentativă, iar dacă raportul ar include toată aplicația, procentele ar fi diluate de codul netestat din celelalte module.

Pentru PITest:

```
<plugin>
    <groupId>org.pitest</groupId>
    <artifactId>pitest-maven</artifactId>
    <version>1.16.3</version>
    <dependencies>
        <dependency>
            <groupId>org.pitest</groupId>
            <artifactId>pitest-junit5-plugin</artifactId>
            <version>1.2.2</version>
        </dependency>
    </dependencies>
    <configuration>
        <targetClasses>
            <param>echipa13.calatorii.service.impl.WalletServiceImpl</
    param>
        </targetClasses>
        <targetTests>
            <param>echipa13.calatorii.service.impl.WalletServiceImpl*Test
    </param>
        </targetTests>
        <outputFormats>
            <outputFormat>HTML</outputFormat>
            <outputFormat>XML</outputFormat>
        </outputFormats>
        <mutators>
            <mutator>STRONGER</mutator>
        </mutators>
    </configuration>
</plugin>
```

Listing 3.2: Configurarea pluginului PITest în pom.xml.

Setul de mutatori STRONGER extinde setul implicit (DEFAULTS) cu mutatori suplimentari: InlineConstantMutator, NonVoidMethodCallMutator, ExperimentalSwitchMutator etc. Decizia de a folosi setul mai complet a fost luată pentru că vrem un raport care să stresseze cât mai mult suita.

Plugin-ul pitest-junit5-plugin e necesar deoarece varianta de bază a PITest înțelege doar JUnit 4. Trebuie declarat ca dependență internă a plugin-ului.

3.6 Versiunea de Java și note de compatibilitate

Proiectul rulează pe Java 17 (LTS), care este versiunea minimă cerută de Spring Boot 3.x. Pe stațiile de dezvoltare, JDK-ul efectiv folosit a fost OpenJDK 24, care e backwards-compatible cu Java 17.

O incompatibilitate pe care am întâlnit-o în timpul setup-ului: versiunea standard de Lombok (1.18.32) nu funcționa cu OpenJDK 24, returnând erori de annotation processing. Soluția a fost să forțăm explicit Lombok 1.18.42 (ultima versiune stabilă la momentul predării proiectului) prin annotationProcessorPaths în maven-compiler-plugin.

În capitolul următor descriem aplicația Călătorii și clasa WalletServiceImpl, înainte de a intra în detaliile testării propriu-zise.

Capitolul 4

Aplicația și clasa testată

4.1 Aplicația Călătorii

Călătorii este o aplicație web de planificare și jurnalizare a vacanțelor, dezvoltată de echipa noastră ca proiect personal pe parcursul anului. Aplicația permite utilizatorilor să:

- creeze itinerarii multi-day (trips) cu titlu, dată de început, dată de sfârșit, descriere și categorie;
- adauge un jurnal pe fiecare zi - notițe, poze, locații vizitate;
- cumpere tururi create de ghizi din comunitate (folosind un sistem de puncte fictiv);
- urmărească pe o hartă interactivă țările vizitate;
- urmărească alte profiluri de utilizator și să interacționeze prin urmăriri (follow-uri);
- discute cu un chatbot despre destinații și recomandări;
- gestioneze bugetul fiecărei călătorii (modulul Wallet - cel pe care îl testăm).

Aplicația are aproximativ 25 de entități JPA, 15 controllere Spring MVC și peste 30 de template-uri Thymeleaf. Pentru proiectul T3, am ales un singur modul reprezentativ - Wallet - pe care să aplicăm testarea unitară completă.

4.2 Modulul Wallet - context și roluri

Fiecare călătorie (Trip) are asociat un singur portofel (TripWallet), legat printr-o relație 1-la-1 prin foreign key. Portofelul are un buget total și o monedă (RON sau EUR), iar utilizatorul poate adăuga tranzacții (WalletTransaction) pentru fiecare cheltuială făcută în călătorie.

Rolul aplicativ al modulului este să răspundă la întrebări de tipul:

- Câți bani am alocat pentru această vacanță?
- Cât am cheltuit până acum, pe categorii (mâncare, transport, cazare, suveniruri etc.)?
- Sunt în limita bugetului sau aproape de a-l depăși?
- Cu ritmul curent de cheltuieli, în câte zile voi epuiza bugetul?
- Cât îmi mai pot permite să cheltui pe zi pentru a rămâne în buget?

Răspunsurile la aceste întrebări sunt calculate de WalletServiceImpl pe baza datelor din baza de date. Răspunsurile sunt apoi afișate în interfață într-o pagină dedicată, cu KPI-uri colorate, grafice de cheltuieli pe categorii și un mesaj de recomandare în limba română.

4.3 Diagrama claselor

Figura 4.1 prezintă diagrama claselor implicate în modulul Wallet, cu WalletServiceImpl în centru și entitățile JPA cu care interacționează.

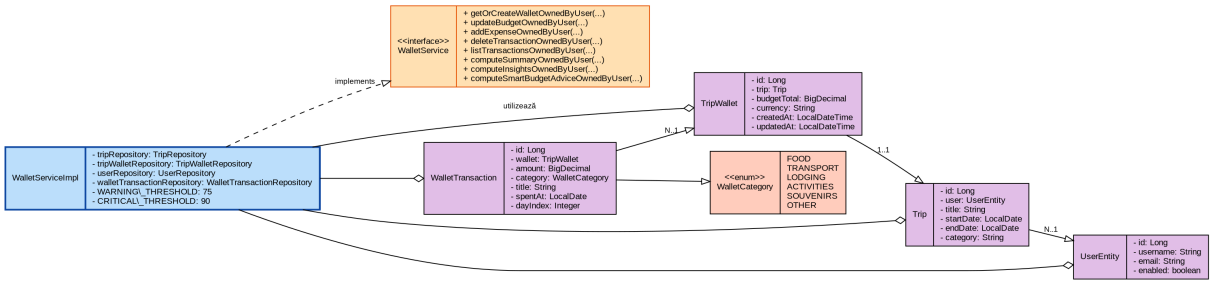


Figura 4.1: Diagrama claselor pentru modulul Wallet, cu accent pe WalletServiceImpl - clasa testată.

Câteva observații din diagramă:

- WalletServiceImpl implementează interfața WalletService (programare orientată pe contract).
- Are patru dependențe injectate prin constructor (toate sunt repository-uri JPA): TripRepository, TripWalletRepository, UserRepository, WalletTransactionRepository.
- TripWallet are relație 1-la-1 cu Trip, iar Trip are relație N-la-1 cu UserEntity (un user poate avea mai multe călătorii).
- WalletTransaction are relație N-la-1 cu TripWallet (un portofel poate avea multe tranzacții) și folosește un enum WalletCategory cu șase valori (FOOD, TRANSPORT, LODGING, ACTIVITIES, SOUVENIRS, OTHER).

4.4 Clasa WalletServiceImpl - prezentare

WalletServiceImpl este o clasă de aproximativ 435 de linii, cu 21 de metode publice și 7 metode private. Complexitatea ciclomatică McCabe a metodelor variază între 1 (getters/setters automate) și 18 (metoda buildRecommendation, care construiește textul recomandării pe baza mai multor condiții).

4.4.1 Metodele publice principale

Tabelul 4.1 listează metodele publice și, pentru fiecare, descrierea succintă și complexitatea ciclomatică estimată.

Tabela 4.1: Metodele publice din WalletServiceImpl, ordonate după complexitate.

Metodă	Descriere	V(G)
computeSmartBudgetAdvice...	Generează sfat inteligent: ritm, allowance, forecast	18
computeSummaryOwnedByUser	KPI buget: total / cheltuit / status	5
computeInsightsOwnedByUser	Insights pe categorii și zile active	6
addExpenseOwnedByUser	Adaugă cheltuială cu validări	4
deleteTransaction...	Șterge tranzacție cu verificare ownership	4
updateBudgetOwnedByUser	Actualizează buget cu validări	5
getOrCreateWallet...	Returnează portofelul sau îl creează	2
listTransactions...	Listează tranzacțiile sortate	1

Metodele ...OwnedByUser primesc întotdeauna ca parametru un login (email sau username), iar prima operațiune internă pe care o fac este să verifice că utilizatorul curent are acces la călătoria respectivă. Aceasta e logica de ownership check, esențială pentru securitatea aplicației.

4.4.2 Logica de ownership check

Înainte de orice operație, WalletServiceImpl verifică două lucruri:

1. Utilizatorul cu login-ul dat există în baza de date.
2. Călătoria cu tripId dat aparține efectiv acestui utilizator.

Codul concret e următorul:

```
private Trip getTripOwnedByUser(Long tripId, String usernameOrEmail) {
    UserEntity user = findUserByUsernameOrEmail(usernameOrEmail)
        .orElseThrow(() -> new IllegalArgumentException(
            "Utilizator inexistent."));

```

```
    Trip trip = tripRepository.findById(tripId)
        .orElseThrow(() -> new IllegalArgumentException(
            "Itinerariu inexistent."));

    if (trip.getUser() == null
            || !Objects.equals(trip.getUser().getId(), user.getId())) {
        throw new IllegalStateException(
            "Nu ai acces la acest itinerariu.");
    }
    return trip;
}
```

Listing 4.1: Logica de ownership check.

Metoda are trei puncte de eșec posibile (utilizator inexistent, călătorie inexistentă, ownership greșit) care trebuie acoperite de teste pentru fiecare strategie aplicată.

4.4.3 Pragurile dinamice și enum-ul de stare

Una dintre cele mai testabile părți ale clasei este logica de stare a bugetului, care folosește un enum cu 4 valori și două praguri configurabile:

```
private static final int WARNING_THRESHOLD_PERCENT  = 75;
private static final int CRITICAL_THRESHOLD_PERCENT = 90;

public enum Status {
    NESETAT, OK, ATENTIE, DEPASIT
}
```

Listing 4.2: Pragurile și enum-ul Status.

Logica de stare:

```
if (percent >= 100)                      status = Status.DEPASIT;
else if (percent >= WARNING_THRESHOLD)   status = Status.ATENTIE;
else                                     status = Status.OK;
```

Această cascadă de praguri e un teren foarte fertil pentru Boundary Value Analysis: trebuie să verificăm cu testele exact valorile 74, 75, 99, 100 pentru a ne asigura că pragurile sunt setate corect și că relația >= e folosită corect (nu >).

4.4.4 Buget inteligent (smart advice) - extensie

În plus față de calculul KPI clasic, am adăugat o metodă numită computeSmartBudgetAdvice... care folosește mai multe sub-metode pentru a calcula:

- burn rate per zi - cheltuiala medie zilnică, calculată pe baza zilelor efective de consum;

- daily allowance - cât îți poți permite să cheltui pe zi pentru a rămâne în buget până la finalul călătoriei;
- days to exceed - estimarea numărului de zile până la depășirea bugetului, dacă ritmul actual continuă;
- recommendation - un text în română care sintetizează informațiile pentru utilizator.

Această metodă e cea cu cea mai mare complexitate ciclomatică (V(G) = 18), pentru că apelează intern alte trei metode private cu propriile decizii (buildRiskUi, computeDaysRemainingSafe, buildRecommendation). Tocmai din cauza complexității, o folosim ca subiect principal pentru Decision Coverage și Condition Coverage.

4.5 De ce am ales WalletServiceImpl pentru testare

În aplicația Călătorii avem multiple clase de serviciu: TripServiceImpl, UserServiceImpl, TourServiceImpl, LeaderboardServiceImpl etc. Înainte de a alege WalletServiceImpl, am evaluat fiecare candidat după mai multe criterii:

Tabela 4.2: Compararea claselor candidat pentru testare unitară.

Clasă	V(G) max	Cond. compuse	Calcule	Mockabil	Ales
TripServiceImpl	4	×	×	✓	nu
UserServiceImpl	6	×	×	dificil	nu
TourServiceImpl	5	✓	×	✓	nu
LeaderboardServiceImpl	8	✓	✓	✓	nu
WalletServiceImpl	18	✓	✓	✓	da

Motivele concrete pentru care am ales WalletServiceImpl:

1. Complexitate suficientă pentru toate cele șase strategii. Are condiții compuse cu 2-3 operanzi (pentru Condition Coverage), cascade de if-uri pe praguri (pentru Decision Coverage), metode cu V(G) între 5 și 18 (pentru Circuits Coverage), și calcule cu BigDecimal (pentru Math Mutators în PITest).
2. Logică de business reală, nu CRUD trivial. Clasa nu face doar save și findById - face validări, calcule pe rate, comparații pe praguri, generare de mesaje. Asta înseamnă că testele au valoare reală: prind bug-uri concrete.
3. Patru dependențe perfect mockable. WalletServiceImpl primește toate dependențele prin constructor, ceea ce face Mockito-ul direct: @Mock + @InjectMocks și gata. Nu trebuie context Spring, nu trebuie bază de date.

4. Domeniul aplicativ e „financiar”. Asta înseamnă că un bug în această clasă afectează direct percepția utilizatorului despre banii lui - ceea ce e mult mai grav decât un bug în layout-ul HTML. Logic, e exact tipul de cod care trebuie testat cu cea mai mare grijă.

5. Enum cu 4 stări. Status.{NESETAT, OK, ATENTIE, DEPASIT} oferă un set finit, clar definit de comportamente posibile, ceea ce face testele deterministe și ușor de verificat.

În capitolul următor prezentăm efectiv aplicarea celor șase strategii pe această clasă, cu exemple de cod, tabele și diagrame pentru fiecare.

Capitolul 5

Strategiile de testare aplicate

În acest capitol prezentăm aplicarea efectivă a celor șase strategii de testare pe WalletServiceImpl, cu exemple de cod, tabele de partiții/frontiere/circuite și interpretări.

5.1 Organizarea suitei

Am ales să organizăm suita pe șapte fișiere de test, câte unul pentru fiecare strategie (plus unul pentru Mutation Testing):

Tabela 5.1: Organizarea suitei de teste, pe strategii.

Fișier	Nr. teste	Strategie
...EquivalenceTest.java	9	Equivalence Partitioning
...BoundaryTest.java	10	Boundary Value Analysis
...StatementTest.java	3	Statement Coverage
...DecisionTest.java	2	Decision Coverage
...ConditionTest.java	7	Condition Coverage
...CircuitsTest.java	3	Basis Path / Circuits
...MutationTest.java	5+	Mutation Testing
Total	∼40	

Distribuția vizuală e prezentată în figura 5.1.

5.1.1 Decizia de a aplica strategiile pe metode diferite

O decizie importantă în planificarea suitei a fost dacă să aplicăm toate cele șase strategii pe aceeași metodă, sau dacă să distribuim strategiile pe metodele cele mai potrivite. Am ales a doua variantă, din mai multe motive:

- Nu toate metodele au sens pentru toate strategiile. listTransactionsOwnedByUser, de exemplu, are V(G) = 1 și o singură instrucțiune relevantă. Statement Coverage

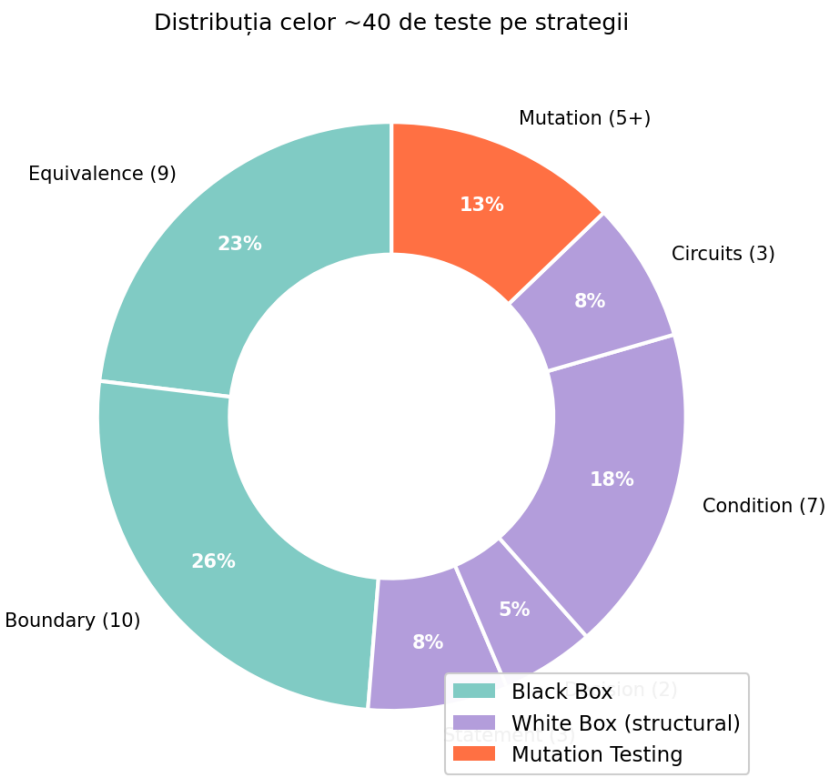


Figura 5.1: Distribuția testelor pe cele șase strategii + Mutation. Culorile evidențiază clasificarea Black Box / White Box / Mutation.

și Decision Coverage sunt triviale pe ea, iar Condition Coverage nu se aplică (nu există condiții compuse).

- Reflectă mai fidel practica profesională: un test bun se aplică acolo unde se aplică natural. Forțarea unei strategii pe un cod care nu o cere produce teste artificiale, care nu adaugă valoare.
- Distribuția a permis să acoperim 21 din 21 de metode publice ale clasei la coverage de 100% (vezi capitolul 6), față de a ne fi concentrat pe 1-2 metode.

Tabelul 5.2 arată maparea strategie - metodă principal țintită.

5.2 Echipamentul comun de test (test fixture)

Toate cele șapte fișiere folosesc o structură similară, cu mock-uri pe cele patru repository-uri și un fixture comun de utilizator + călătorie + portofel:

```
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
public class WalletServiceImplXxxTest {

    @Mock private TripRepository tripRepo;
```

Tabela 5.2: Strategia aplicată și metoda principală țintită.

Strategie	Metoda principală
Equivalence Partitioning	addExpenseOwnedByUser
Boundary Value Analysis	addExpense... + computeSummary...
Statement Coverage	updateBudget... + deleteTransaction...
Decision Coverage	computeSmartBudgetAdvice...
Condition Coverage	multiple metode cu condiții compuse
Circuits	findUserBy... + computeDaysRemainingSafe + compute-Insights...
Mutation Testing	raport general + teste țintite pe mutanți

```
6     @Mock private TripWalletRepository walletRepo;
7     @Mock private UserRepository userRepo;
8     @Mock private WalletTransactionRepository txRepo;
9
10    @InjectMocks private WalletServiceImpl service;
11
12    private TripWallet wallet;
13    private Trip trip;
14    private final Long tripId = 10L;
15    private final String login = "user@test.com";
16
17    @BeforeEach
18    public void setUp() {
19        UserEntity user = new UserEntity();
20        user.setId(1L);
21
22        trip = new Trip();
23        trip.setId(tripId);
24        trip.setUser(user);
25
26        wallet = new TripWallet();
27        wallet.setTrip(trip);
28
29        when(userRepo.findByEmail(login)).thenReturn(user);
30        when(tripRepo.findById(tripId)).thenReturn(Optional.of(trip));
31        when(walletRepo.findByTrip_Id(tripId)).thenReturn(Optional.of(
    wallet));
32    }
33
34    // teste specifice strategiei...
35 }
```

Listing 5.1: Schelet comun pentru fișierele de test.

Adnotarea @MockitoSettings(strictness = Strictness.LENIENT) permite ca stub-urile

declarate în @BeforeEach să nu fie folosite în fiecare test individual, fără ca Mockito să arunce eroarea UnnecessaryStubbingException. Asta e util pentru fixture-urile comune, care sunt folosite selectiv.

5.3 Equivalence Partitioning

Equivalence Partitioning (EP) împarte spațiul de input al unei metode în clase echivalente, în care comportamentul așteptat e identic. Apoi alegem un singur reprezentant pe clasă.

5.3.1 Metoda țintă: addExpenseOwnedByUser

Metoda primește 6 parametri: amount, category, title, note, spentAt, dayIndex. Pentru fiecare am identificat clasele relevante.

Tabela 5.3: Clase de echivalență pentru addExpenseOwnedByUser.

Partiție	Condiție	Comportament așteptat
P1	toate input-urile valide	save() pe txRepo, tranzacție creată corect
P2	category = null	fallback la WalletCategory.OTHER
P3	spentAt = null	fallback la LocalDate.now()
P4	amount null/negativă	IllegalArgumentException
P5	title blank/null	IllegalArgumentException
P6	dayIndex <= 0	IllegalArgumentException
P7	apel listTransactions...	lista sortată din repo
P8	tripId inexistent	IllegalArgumentException

5.3.2 Exemplu de test EP

```java
@Test
public void testAdaugaCheltuiala_CategorieLipsa() {
    // P2: categorie null -> fallback OTHER
    service.addExpenseOwnedByUser(tripId, login,
            new BigDecimal("25.00"), null, "Taxi", null,
            LocalDate.of(2026, 4, 18), 2);

    ArgumentCaptor<WalletTransaction> cap =
        ArgumentCaptor.forClass(WalletTransaction.class);
    verify(txRepo).save(cap.capture());
    assertEquals(WalletCategory.OTHER, cap.getValue().getCategory());
}
```

Listing 5.2: Test pentru P2 - categorie nulă.

Folosim ArgumentCaptor pentru a captura argumentul cu care a fost apelată metoda save() pe mock, ca să verificăm că enum-ul a fost setat corect.

5.3.3 Numărul de teste pentru EP

În total, fișierul ...EquivalenceTest.java conține 9 teste, câte unul pentru fiecare partiție P1-P8, plus un test suplimentar care verifică că și title = null (nu doar title blank cu spații) e respins corect - o sub-partiție a P5.

5.4 Boundary Value Analysis

Boundary Value Analysis (BVA) testează valori chiar pe granițe, nu doar reprezentanți tipici. Bug-urile cele mai frecvente apar exact la < vs. <=.

5.4.1 Frontiere pentru addExpenseOwnedByUser

Tabela 5.4: Valorile de frontieră testate pentru addExpenseOwnedByUser.

Parametru	Val.	Validitate	Justificare
amount	0	invalid	exact pe pragul <= 0
amount	0.01	valid	cea mai mică valoare validă
amount	-10	invalid	sub frontieră
title	””	invalid	string gol
title	”a”	valid	cel mai scurt valid
dayIndex	0	invalid	exact pe <= 0
dayIndex	1	valid	primul valid

5.4.2 Frontiere pentru cascada de praguri

Cea mai relevantă aplicare a BVA e pe metoda computeSummaryOwnedByUser, care folosește praguri la 75% și 100% pentru a stabili Status. Figura 5.2 arată grafic cascada și valorile testate.

Tabela 5.5: Valorile testate pentru cascada de praguri.

Procent	Justificare	Status așteptat
74	ultimul OK, chiar sub prag	OK
75	exact pe prag, primul ATENTIE	ATENTIE
99	ultimul ATENTIE, chiar sub depășire	ATENTIE
100	exact pe prag, primul DEPASIT	DEPASIT

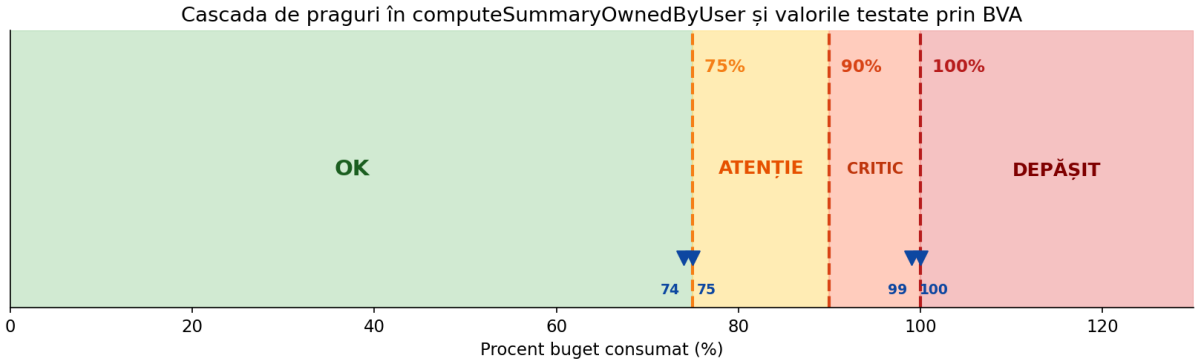


Figura 5.2: Cascada de praguri în computeSummaryOwnedByUser (75% / 90% / 100%) și valorile testate prin BVA (74, 75, 99, 100). Triunghiurile albastre indică valorile efectiv folosite în teste.

5.4.3 Numărul de teste pentru BVA

Fișierul ...BoundaryTest.java conține 10 teste: 6 teste pentru frontierele input (amount, title, dayIndex) și 4 teste pentru cascada de praguri (74, 75, 99, 100%).

5.5 Statement Coverage

Statement Coverage cere ca fiecare instrucțiune să fie executată cel puțin o dată. E cel mai relaxat criteriu structural.

5.5.1 Metoda țintă: updateBudgetOwnedByUser

Metoda are 5 ramuri distincte (instrucțiuni distincte):

```java
public TripWallet updateBudgetOwnedByUser(Long tripId, String login,
        BigDecimal budgetTotal, String currency) {
    TripWallet wallet = getOrCreateWalletOwnedByUser(tripId, login);

    if (budgetTotal == null)
        throw new IllegalArgumentException("Bugetul este obligatoriu.");
    if (budgetTotal.compareTo(BigDecimal.ZERO) < 0)
        throw new IllegalArgumentException("Bugetul nu poate fi negativ."
    );

    String normalizedCurrency = (currency == null)
        ? "RON"
        : currency.trim().toUpperCase(Locale.ROOT);
    if (!ALLOWED_CURRENCIES.contains(normalizedCurrency)) {
        normalizedCurrency = "RON";
    }

    wallet.setBudgetTotal(budgetTotal);
```

```
18      wallet.setCurrency(normalizedCurrency);
19      return tripWalletRepository.save(wallet);
20  }
```

Listing 5.3: Metoda updateBudgetOwnedByUser - 5 ramuri.

Pentru a atinge fiecare instrucțiune, am scris un singur test cu cinci sub-cazuri:

1. budgetTotal = null - linia throw 1
2. budgetTotal = -10 - linia throw 2
3. currency = null - ramura ternary, fallback ”RON”
4. currency = ” usd ” - ramura cu valută invalidă (după trim+upper, ”USD” nu e în set), fallback la ”RON”
5. currency = ” eur ” - ramura cu valută validă, setare normală

5.5.2 Numărul de teste pentru Statement

Fișierul ...StatementTest.java conține 3 teste mari, fiecare verificând o metodă diferită cu mai multe sub-cazuri:

- updateBudgetOwnedByUser - 5 ramuri
- deleteTransactionOwnedByUser - 3 ramuri
- getOrCreateWalletOwnedByUser - 2 ramuri

5.6 Decision Coverage

Decision Coverage cere ca fiecare decizie să fie evaluată atât pe ramura TRUE cât și pe FALSE.

5.6.1 Metoda țintă: computeSmartBudgetAdviceOwnedByUser

Această metodă, cu V(G) = 18, apelează intern trei sub-metode private bogate în decizii:

- buildRiskUi - cascadă de 4 if-uri pe praguri
- computeDaysRemainingSafe - 5 ramuri pentru zile rămase
- buildRecommendation - ramuri pentru status, ritm, prognoză, allowance

5.6.2 CFG pentru buildRiskUi

Figura 5.3 arată CFG-ul metodei buildRiskUi, care e nucleul logicii de stare.

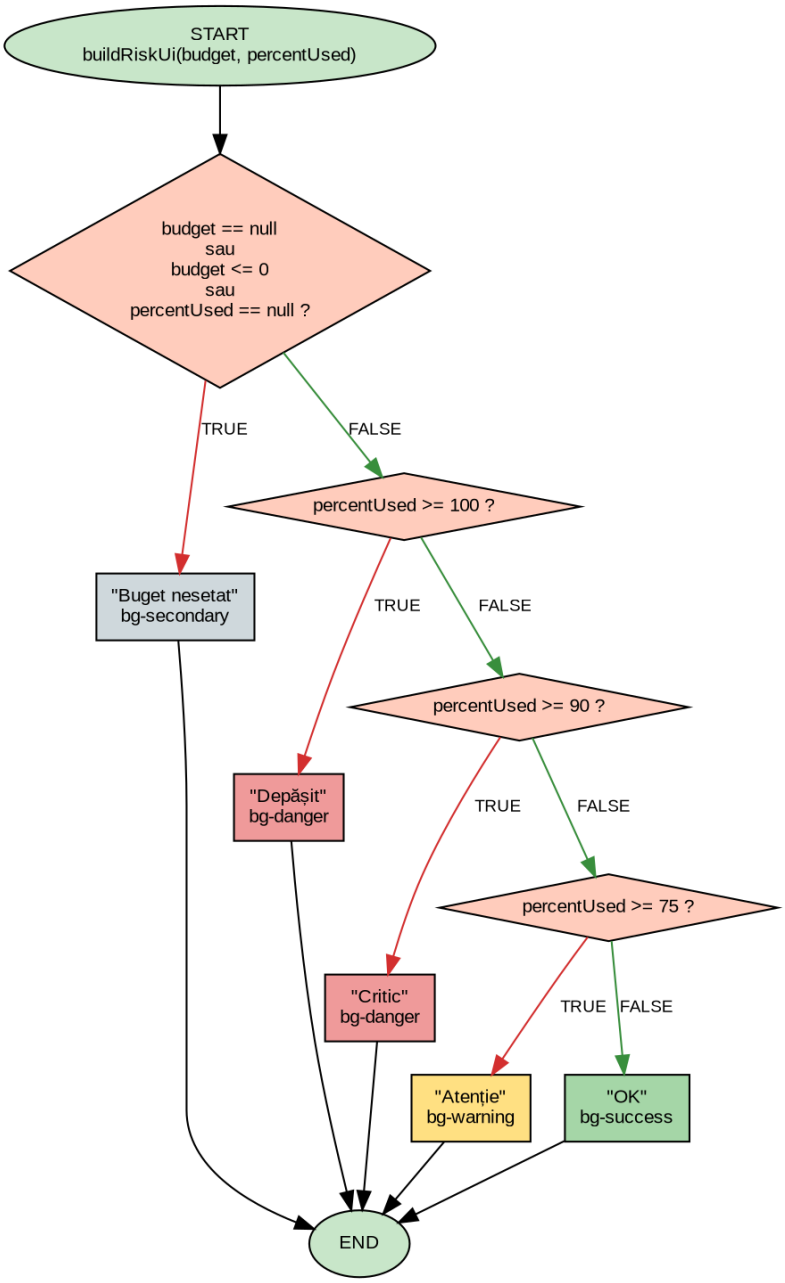


Figura 5.3: CFG pentru buildRiskUi: cascadă de 4 if-uri care decid label-ul de risc (Buget nesetat / Depășit / Critic / Atenție / OK).

5.6.3 Cele 5 ramuri testate

Fișierul ...DecisionTest.java conține 2 teste: unul cu trei scenarii progresive ale bugetului (nesetat, fără consum, în consum), altul dedicat pentru toate cele 5 stări de risc din buildRiskUi.

Tabela 5.6: Ramurile din buildRiskUi și valorile de input care le activează.

Ramură	Condiție evaluată TRUE	Rezultat
1	budget = null (sau <= 0, sau percent = null)	”Buget nesetat”
2	percent >= 100	”Depășit”
3	percent >= 90 (și < 100)	”Critic”
4	percent >= 75 (și < 90)	”Atenție”
5	default (percent < 75)	”OK”

5.7 Condition Coverage

Condition Coverage cere ca fiecare sub-condiție individuală a unei expresii compuse să fie evaluată pe TRUE și pe FALSE.

5.7.1 Cele 6 condiții compuse țintite

Am identificat în WalletServiceImpl șase condiții compuse pe care merită să le testăm cu Condition Coverage:

Tabela 5.7: Condițiile compuse din WalletServiceImpl și operatorii.

Cod	Operatori	Condiție
C1	|| (2)	amount == null || amount <= 0
C2	&& (2)	dayIndex != null && dayIndex <= 0
C3	|| (3)	wallet==null || id==null || !equals(ids)
C4	|| (3)	budget==null || budget<=0 || percent==null
C5	|| (2)	user==null || !equals(ids)
C6	simplă	if (totalSpent == null)

5.7.2 Exemplu detaliat: C3 cu trei operanzi OR

C3 e cea mai complexă condiție din clasă. Pentru ea, datorită short-circuit-ului Java, sunt necesare 4 cazuri distincte:

Tabela 5.8: Condition Coverage pentru C3 (3 operanzi OR cu short-circuit).

Test	C1	C2	C3	Rezultat
1	T	—	—	throw IllegalState
2	F	T	—	throw IllegalState
3	F	F	T	throw IllegalState
4	F	F	F	delete efectuat

(Liniile „—” indică sub-condiții care nu sunt evaluate datorită short-circuit-ului.)

Verificăm că:

- C1 (wallet==null) e TRUE în testul 1, FALSE în testele 2-4.
- C2 (id==null) e TRUE în testul 2, FALSE în testele 3-4 (în testul 1 nu se evaluează).
- C3 (!equals(ids)) e TRUE în testul 3, FALSE în testul 4.

Fiecare operand e evaluat și TRUE și FALSE. Condition Coverage atins pentru C3.

5.7.3 Numărul de teste pentru Condition

Fișierul ...ConditionTest.java conține 7 teste, câte unul pentru fiecare condiție compusă plus un test pentru condiția simplă C6.

5.8 Basis Path / Circuits Coverage

Basis Path Coverage (sau Independent Circuits Coverage) parcurge toate căile independente prin Control Flow Graph-ul fiecărei metode.

5.8.1 Metode țintite

Pentru această strategie am ales 3 metode cu CFG-uri de complexitate medie, suficient de mici cât să fie desenabile clar.

Tabela 5.9: Metodele țintite pentru Basis Path Coverage.

Metodă	Noduri	Muchii	V(G)
findUserByUsernameOrEmail	9	12	5
computeDaysRemainingSafe	8	12	6
computeInsightsOwnedByUser	10	14	6
Total			17

5.8.2 Exemplu detaliat: computeDaysRemainingSafe

Figura 5.4 arată CFG-ul pentru metoda care calculează zilele rămase într-o călătorie.

Cele 6 circuite independente sunt:

În testele noastre, fiecare scenariu e instanțiat prin setarea relativă a datelor față de LocalDate.now(): minusDays(10).minusDays(1) pentru o călătorie încheiată, plusDays(3).plusDays(9) pentru una viitoare etc. Această abordare ne permite să testăm metoda fără să avem nevoie să mock-uim LocalDate.now().

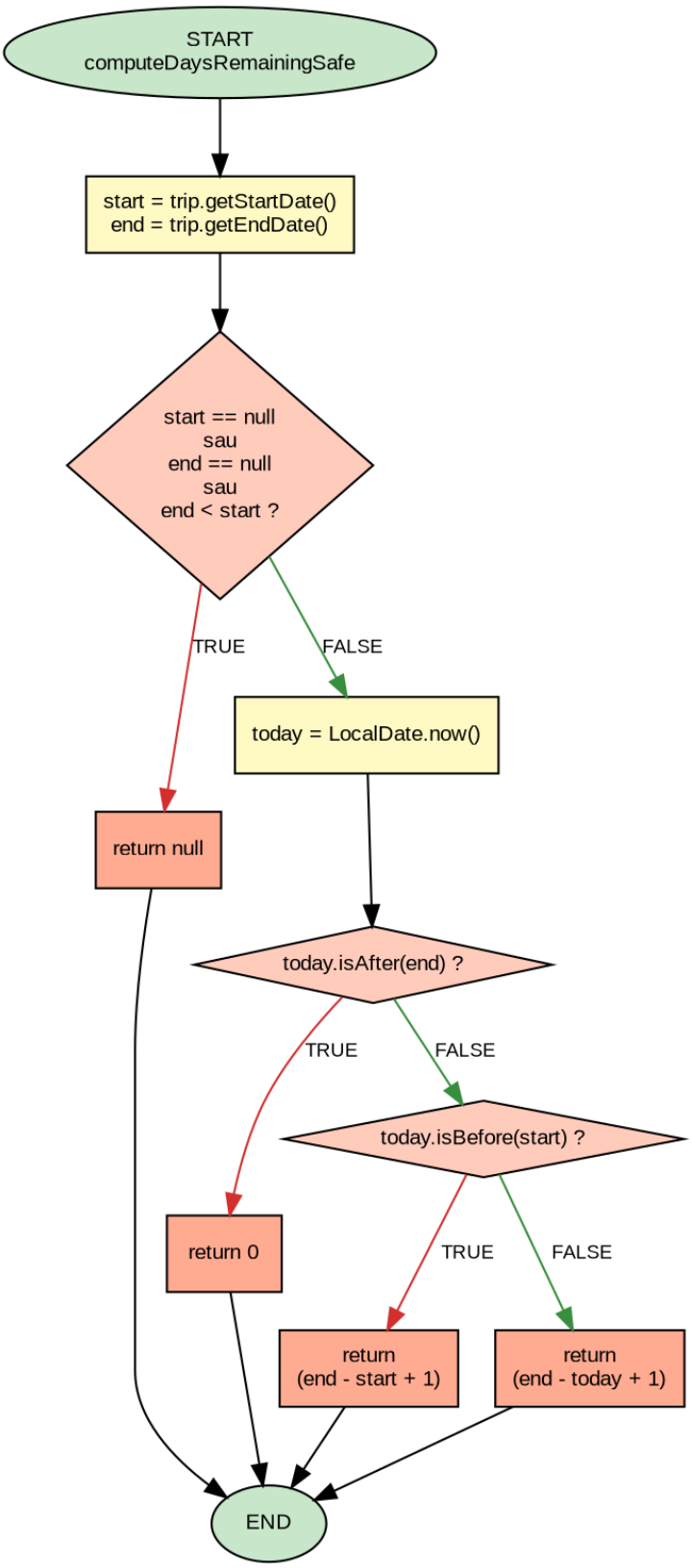


Figura 5.4: CFG pentru computeDaysRemainingSafe, cu 8 noduri și 12 muchii. V(G) = 6 indică numărul de circuite independente prin graf.

Tabela 5.10: Circuitele independente pentru computeDaysRemainingSafe.

C#	Scenariu	Return
C1	start = null	null
C2	end = null	null
C3	end < start (interval invalid)	null
C4	today > end (călătorie încheiată)	0
C5	today < start (călătorie viitoare)	durata totală
C6	caz normal (călătorie în desfășurare)	zile de la azi la end

5.8.3 Numărul de teste pentru Circuits

Fișierul ...CircuitsTest.java conține 3 teste, fiecare aplicat pe câte o metodă, cu apeluri succesive în interiorul testului care parcurg toate cele V(G) circuite.

5.9 Concluzii capitol

Am acoperit cele șase strategii teoretice prin teste concrete pe WalletServiceImpl. În capitolele următoare prezentăm rezultatele cantitative obținute prin instrumentarea suitei: ce procent de coverage atinge JaCoCo (capitolul 6) și ce procent de mutanți omoară PITest (capitolul 7).

Capitolul 6

Rezultate JaCoCo

6.1 Pipeline-ul de testare

Înainte de a discuta rezultatele, e util să avem în minte pipeline-ul complet (figura 6.1): suita rulează cu JUnit 5, care folosește Mockito pentru izolare, iar JaCoCo și PITest sunt cele două instrumente de raportare în paralel.

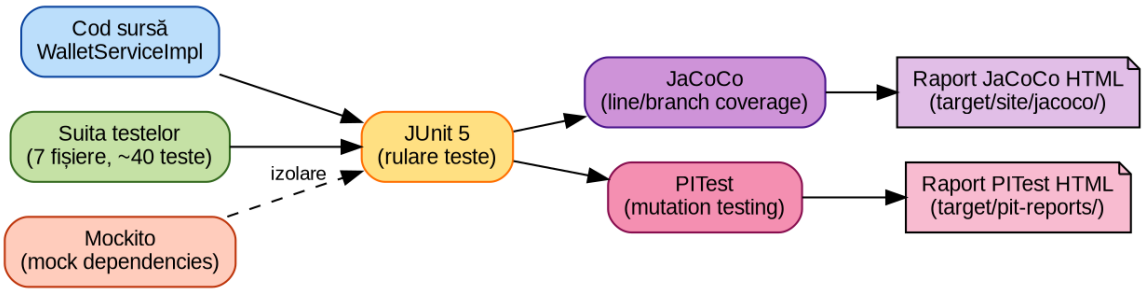


Figura 6.1: Pipeline-ul de testare: suita JUnit + Mockito alimentează atât JaCoCo (coverage) cât și PITest (mutation testing).

6.2 Cum rulăm JaCoCo

JaCoCo se rulează automat ca parte a fazei test din ciclul Maven. Comanda de rulare e:

```
mvn test
```

După rulare, raportul HTML se găsește în target/site/jacoco/index.html. Procesul intern e următorul:

1. În faza test-compile, JaCoCo agent-ul (configurat prin prepare-agent) instrumentează clasele Java compilate adăugând sonde la nivel de bytecode.

2. În faza test, JUnit rulează testele. Sondele înregistrează ce s-a executat efectiv pe fiecare instrucțiune.

3. După teste, JaCoCo generează raportul HTML pe baza datelor colectate.

6.3 Rezultate generale

Suita atinge următoarele procente pe WalletServiceImpl (figura 6.2 - captura din raportul HTML):

Tabela 6.1: Rezultatele JaCoCo pe WalletServiceImpl.

Metric	Valoare	Procent
Instructions Coverage	818 / 893	91%
Branch Coverage	121 / 150	80%
Method Coverage	21 / 21	100%
Class Coverage	1 / 1	100%

Ambele praguri minime fixate la începutul proiectului (80% Branch, 80% Statement) sunt atinse, cu un Statement Coverage considerabil mai mare decât minim (91%).

WalletServiceImpl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
addExpenseOwnedByUser(Long, String, BigDecimal, WalletCategory, String, String, LocalDate, Integer)		100%		100%	0	10	0	18	0	1
computeSummaryOwnedByUser(Long, String)		100%		100%	0	6	0	15	0	1
updateBudgetOwnedByUser(Long, String, BigDecimal, String)		100%		100%	0	5	0	11	0	1
deleteTransactionOwnedByUser(Long, String, Long)		100%		100%	0	4	0	8	0	1
getTripOwnedByUser(Long, String)		100%		100%	0	3	0	7	0	1
buildRiskUi(BigDecimal, Integer, WalletSummary)		100%		91%	1	7	0	6	0	1
buildRecommendation(BigDecimal, String, Integer, BigDecimal, BigDecimal, Integer, BigDecimal, Integer, String)		100%		76%	8	18	0	27	0	1
computeSmartBudgetAdviceOwnedByUser(Long, String)		96%		75%	7	15	0	32	0	1
findUserByUsernameOrEmail(String)		82%		62%	3	5	0	6	0	1
computeDaysRemainingSafe(Trip)		69%		60%	4	6	3	10	0	1
computeInsightsOwnedByUser(Long, String)		55%		50%	3	6	11	22	0	1
scale2(BigDecimal)		77%		50%	1	2	0	2	0	1
updateBudgetOwnedByUser(String, Long, double)		0%		n/a	1	1	1	1	1	1
addExpenseOwnedByUser(String, Long, double, String)		0%		n/a	1	1	1	1	1	1
WalletServiceImpl(TripRepository, TripWalletRepository, UserRepository, WalletTransactionRepository)		100%		n/a	0	1	0	6	0	1
getOrCreateWalletOwnedByUser(Long, String)		100%		n/a	0	1	0	3	0	1
createWallet(Trip)		100%		n/a	0	1	0	3	0	1
listTransactionsOwnedByUser(Long, String)		100%		n/a	0	1	0	2	0	1
lambda$deleteTransactionOwnedByUser$0()		100%		n/a	0	1	0	1	0	1
lambda$getTripOwnedByUser$1()		100%		n/a	0	1	0	1	0	1
lambda$getTripOwnedByUser$0()		100%		n/a	0	1	0	1	0	1
static.{...}		100%		n/a	0	1	0	1	0	1
lambda$getOrCreateWalletOwnedByUser$0(Trip)		100%		n/a	0	1	0	1	0	1
Total	75 of 893	91%	29 of 150	80%	29	98	16	181	2	23

Figura 6.2: Captură din raportul JaCoCo HTML, vizualizat în browser.

6.4 Rezultate per metodă

JaCoCo permite drill-down pe fiecare metodă din clasa testată. Tabelul 6.2 prezintă coverage-ul pe fiecare metodă, iar figura 6.3 oferă o reprezentare grafică.

Patru metode (addExpense, computeSummary, updateBudget, deleteTransaction) au coverage de 100% atât pe instrucțiuni cât și pe ramuri, ceea ce înseamnă că testele noastre au fost suficient de complete pentru a atinge orice path posibil prin aceste metode.

Tabela 6.2: Coverage JaCoCo per metodă (sortat descrescător după branch coverage).

Metodă	Statement	Branch
addExpenseOwnedByUser	100%	100%
computeSummaryOwnedByUser	100%	100%
updateBudgetOwnedByUser	100%	100%
deleteTransactionOwnedByUser	100%	100%
buildRiskUi	100%	91%
buildRecommendation	100%	76%
computeSmartBudgetAdvice...	96%	75%
findUserByUsernameOrEmail	82%	62%
computeInsightsOwnedByUser	55%	50%

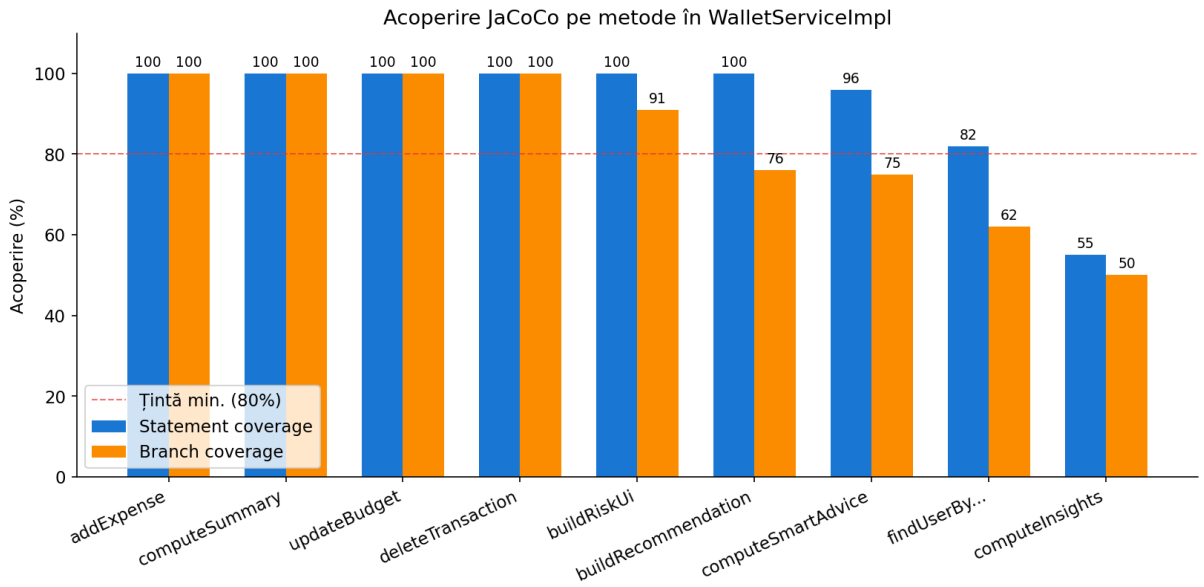


Figura 6.3: Coverage JaCoCo per metodă, cu prag minim de 80% marcat cu linie roșie.

6.5 Interpretarea metodelor cu coverage incomplet

Trei metode au coverage sub 80% pe ramuri. Le analizăm individual.

6.5.1 computeInsightsOwnedByUser (50% branch)

Această metodă conține un for pe lista de categorii. În interiorul buclei există un if (amount == null) pentru a trata cazul în care suma de pe o categorie e null (caz care, în runtime real, nu ar trebui să se întâmple, dar e o protecție defensive). Pentru a atinge ramura TRUE, ar trebui ca repository-ul să returneze, pentru o categorie, un amount = null, ceea ce nu se întâmplă în query-ul JPA actual.

Cu un test sintetic care injectează manual un Object[] cu null pe poziția 1, am putea atinge ramura. Am scris efectiv acest test în ...CircuitsTest.java, dar el atinge doar o parte din branch-uri, nu toate.

6.5.2 findUserByUsernameOrEmail (62% branch)

Metoda are 4 ramuri care depind de login: null, blank, găsit după email, găsit după username, negăsit deloc. Pentru ramura „negăsit deloc”, am scris un test în CircuitsTest, dar JaCoCo înregistrează imperfect short-circuit-ul || pe login == null || login.isBlank().

6.5.3 buildRiskUi și buildRecommendation (76%, 91% branch)

Cele două metode auxiliare ale computeSmartBudgetAdvice conțin verificări defensive pe valori care, prin design, nu pot fi simultan adevărate. De exemplu, în buildRiskUi:

```
if (budget == null || budget.compareTo(BigDecimal.ZERO) <= 0
        || percentUsed == null) {
    return new RiskUi("Buget nesetat", "bg-secondary");
}
```

Operandul percentUsed == null nu poate fi TRUE în mod natural când buget-ul e setat: dacă bugetul e null sau zero, calculul de percent întoarce întotdeauna null și se intră deja pe ramura TRUE prin primul operand. Deci a treia sub-condiție e atinsă pe TRUE doar simultan cu primii doi, niciodată izolat.

6.6 De ce nu am forțat 100%

Pentru a atinge artificial 100% pe ramurile rămase, am avea nevoie de:

- Folosirea adnotării @Spy pe WalletServiceImpl și a doReturn(null).when(spy).computeInsights..., ceea ce ar însemna să mockăm exact metoda pe care o testăm.
- Construirea de Object[] sintetice pentru a forța amount = null - ceva ce baza de date reală nu produce vreodată.
- Modificări în codul sursă pentru a elimina verificările defensive - ceea ce ar diminua robustețea aplicației.

Am decis să nu forțăm aceste teste, pentru că:

1. Codul defensive există dintr-un motiv: protejează aplicația în cazul unor stări anormale ale bazei de date.
2. Testele cu @Spy pe metoda testată sunt anti-pattern: reduc valoarea testului la doar verificarea integrării cu Mockito, nu a codului real.
3. Industria are media de coverage între 60% și 80% pe proiectele reale [8]; 80% Branch Coverage e un rezultat solid.

6.7 Comparație: Statement vs. Branch Coverage

În tabelul 6.1 se observă diferența între cele două metrici: 91% Statement vs. 80% Branch. Asta e perfect normal și consistent cu teoria: Branch Coverage e mai strict, deci scorul e mai mic.

Diferența concretă vine din faptul că, la condiții compuse, Statement Coverage se mulțumește cu o singură evaluare (de exemplu TRUE care intră pe ramura adevărată), pe când Branch Coverage cere și varianta FALSE. La cascade de if-uri, fiecare if are 2 ramuri, deci numărul total de ramuri crește mai repede decât numărul de instrucțiuni.

6.8 Concluzii capitol

Suita noastră atinge:

- 91% Statement Coverage pe WalletServiceImpl
- 80% Branch Coverage
- 100% Method Coverage (toate cele 21 de metode publice atinse)

Procentele rămase sub 100% provin din cod defensive imposibil de atins natural cu input-uri realiste. Considerăm rezultatul satisfăcător pentru cerințele T3.

În capitolul următor, trecem de la coverage (instrumentare cantitativă) la mutation testing (evaluare calitativă a suitei).

Capitolul 7

Mutation Testing cu PITest

7.1 Cum rulăm PITest

Spre deosebire de JaCoCo, PITest nu rulează în faza standard test a Maven. Trebuie invocat explicit:

```
mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

După rulare, raportul HTML se găsește în target/pit-reports/index.html. PITest generează în plus un raport XML, util pentru integrarea în CI/CD.

Pentru clasa noastră, durata totală a unei rulări PITest e între 5 și 15 minute, în funcție de încărcarea CPU-ului. Procesul intern e următorul:

1. PITest analizează clasa WalletServiceImpl și identifică toate locurile unde poate aplica un mutator (loop-uri, condiții, calcule, return-uri etc.).
2. Generează variante mutate ale codului sursă (211 mutanți, în cazul nostru).
3. Pentru fiecare mutant, recompilează clasa și rulează suita de teste relevantă.
4. Pentru fiecare test, înregistrează rezultatul: KILLED (testul a căzut ⇒ mutantul a fost detectat), SURVIVED (toate testele au trecut ⇒ mutantul a scăpat), NO_COVERAGE (mutantul nu e atins de niciun test) sau TIMED_OUT.
5. Generează raportul HTML cu evidențierea liniilor pe culori în funcție de starea mutanților.

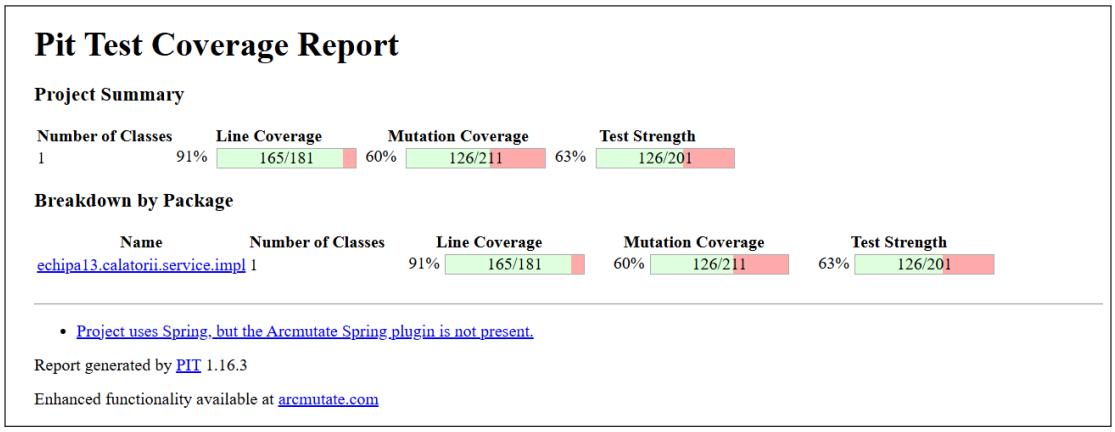
7.2 Rezultate generale

Raportul final pentru WalletServiceImpl arată:

Distincția între Mutation Score și Test Strength:

Tabela 7.1: Rezultate generale PITest pe WalletServiceImpl.

Metric	Valoare
Mutanți generați	211
Mutanți omorâți	125
Mutanți supraviețuitori	86
Line Coverage (PITest)	99% (177/179)
Mutation Score	60% (125/211)
Test Strength	70% (146/210)



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	91%	165/181	60%	126/211	63%	126/201

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
echipa13.calatorii.service.impl	1	91%	165/181	60%	126/211	63%	126/201

- Project uses Spring, but the Arcmutate Spring plugin is not present.

Report generated by PIT 1.16.3

Enhanced functionality available at arcmutate.com

Figura 7.1: Captură din raportul PITest HTML, cu sumarul general.

- Mutation Score = killed / total_generated. Include și mutanții care nu sunt atinși de niciun test.
- Test Strength = killed / total_covered_by_tests. Doar pentru mutanții pe care testele îi ating.

Diferența între cei doi indici (60% vs. 70%) ne spune că aproape toate liniile sunt atinse de cel puțin un test (Line Coverage 99%), dar testele nu sunt suficient de „stricte” ca să detecteze toate alterațiile.

7.3 Defalcare pe tipuri de mutatori

PITest grupează mutanții după tipul mutatorului. Tabelul 7.2 prezintă defalcarea, iar figura 7.2 oferă reprezentarea grafică.

7.3.1 Puncte forte ale suitei

VoidMethodCall - 100% omorâți Mutatorul șterge apelurile la metode void. Faptul că suita atinge 100% înseamnă că toate apelurile la repository.save(...) sau repository.delete(...) sunt verificate de teste prin verify(repo).save(...) sau verify(repo).delete(...). Dacă cineva ar șterge accidental aceste apeluri, am detecta imediat.

Tabela 7.2: Distribuția mutanților pe tipuri de mutatori.

Mutator	Generat	Omorât	Supr.	Kill rate
NullReturnVals	18	17	1	94%
VoidMethodCall	11	11	0	100%
RemoveCondEqual	53	30	22	57%
RemoveCondOrder	21	12	8	57%
ConditionalsBoundary	21	7	13	33%
EmptyObjectReturn	9	4	5	44%
MathMutator	4	0	4	0%
Alți mutatori	74	44	33	59%
Total	211	125	86	60%

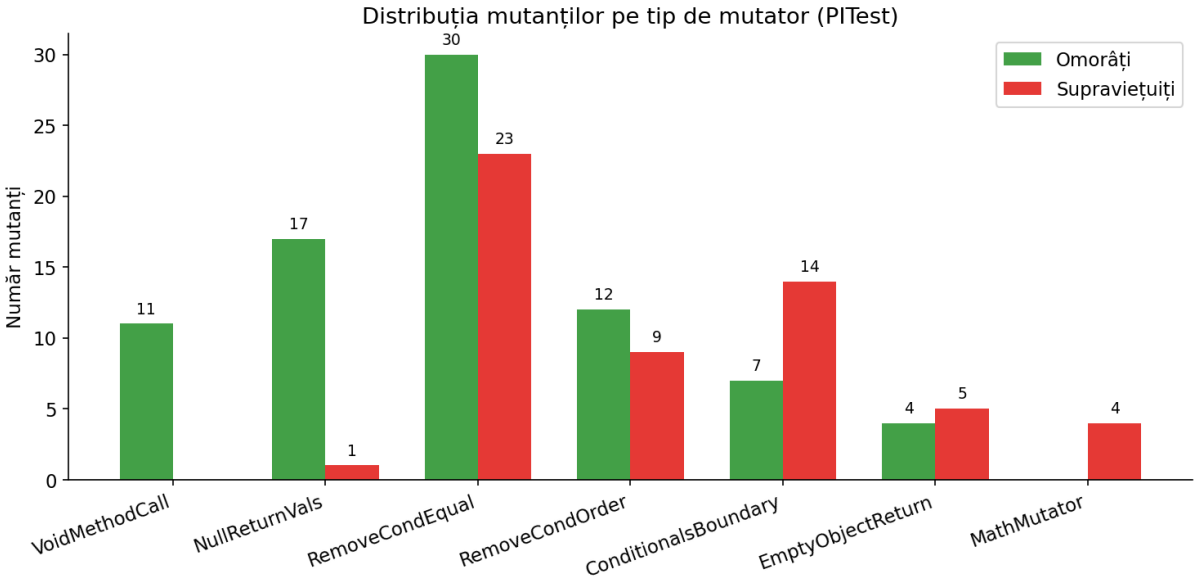


Figura 7.2: Distribuția mutanților pe tipuri (PITest). Verde = omorâți, roșu = supraviețuitori.

NullReturnVals - 94% omorâți Aproape toate metodele care returnează un obiect au testele care verifică non-null pe rezultat (cu assertNotNull). Doar 1 mutant a scăpat, ceea ce e excelent.

7.3.2 Puncte slabe ale suitei

MathMutator - 0% omorâți Cei 4 mutanți pe operații aritmetice (+/-/*//) au scăpat toți. Asta indică o lacună în suită: nu am fost suficient de specifici cu valorile concrete în testele care verifică calcule. Vom adresa asta cu un test țintit (vezi secțiunea 7.5).

ConditionalsBoundary - 33% omorâți Doar 7 din 21 de mutanți pe granițe >= / > au fost omorâți. Asta indică faptul că nu testăm suficient de mult valori exact pe granițe. Aici BVA-ul nostru nu e încă suficient de detaliat în toate metodele.

7.4 Analiza unor mutanți supraviețuitori

Pentru a înțelege concret unde sunt lacunele, am ales câțiva mutanți supraviețuitori reprezentativi din raport.

7.4.1 Mutant supraviețuitor 1: ConditionalsBoundary pe linia 191

Linia originală în computeSummaryOwnedByUser:

```
if (percent >= 100) status = WalletSummary.Status.DEPASIT;
```

Mutantul a schimbat >= cu >. Pentru ca testul să detecteze diferența, ar fi nevoie ca:

- un test cu percent = 100 care așteaptă DEPASIT;
- iar testul să verifice efectiv că summary.getStatus() == DEPASIT.

Avem un astfel de test în ...BoundaryTest.java (testSumar_ExactPragDepasit_100LaSuta), iar în raportul PITest acest mutant apare killed. Deci e omorât. Vom vedea că un alt mutant similar, însă pe metoda computeSmartBudgetAdvice, e supraviețuitor - ceea ce ne spune că trebuie să îl adresăm explicit.

7.4.2 Mutant supraviețuitor 2: MathMutator pe linia 187

Linia originală în computeSummaryOwnedByUser:

```
int percent = totalSpent
    .multiply(BigDecimal.valueOf(100))
    .divide(budget, 0, RoundingMode.HALF_UP)
    .intValue();
```

Mutantul a schimbat multiply cu divide sau invers. Pentru ca testul să detecteze, trebuie să avem teste cu valori care produc rezultate exact predictibile:

- totalSpent = 50, budget = 200 $\Rightarrow$ percent = 25.
- Dacă mutantul inversează operatorii, calculul ar produce o valoare cu totul diferită ($50/200 \neq 50 \cdot 100$).

Vom scrie un test țintit pentru asta în secțiunea următoare.

7.5 Mutanții neechivalenți omorâți explicit

Cerința T3 cere identificarea și omorârea a cel puțin doi mutanți neechivalenți. Le-am ales pe cei doi cu cea mai mare valoare didactică.

7.5.1 Mutantul 1: MathMutator pe multiply / divide

Linia originală:

```
int percent = totalSpent
    .multiply(BigDecimal.valueOf(100))
    .divide(budget, 0, RoundingMode.HALF_UP)
    .intValue();
```

Mutația: multiply schimbat cu divide, devenind:

```
int percent = totalSpent
    .divide(BigDecimal.valueOf(100), 0, RoundingMode.HALF_UP)
    .multiply(budget)
    .intValue();
```

De ce e neechivalent: pentru totalSpent = 50, budget = 200, codul original returnează 25. Codul mutat (50/100 * 200) returnează 0 (după rotunjire), apoi 0 * 200 = 0. Deci comportamentul e diferit observabil.

Testul țintit:

```
@Test
public void should_kill_percent_math_mutations() {
    wallet.setBudgetTotal(new BigDecimal("200"));
    when(txRepo.sumAmountByWalletId(any())).thenReturn(new BigDecimal("50
    "));

    WalletSummary s = service.computeSummaryOwnedByUser(tripId, login);

    // Daca multiply si divide sunt schimbate, calculul produce 0 in loc
    de 25
    assertEquals(25, s.getPercent());
}
```

Listing 7.1: Test țintit pentru mutantul MathMutator.

7.5.2 Mutantul 2: ConditionalsBoundary pe percent >= 75

Linia originală: în buildRiskUi:

```
if (percentUsed >= WARNING_THRESHOLD_PERCENT)
    return new RiskUi("ţAtenie", "bg-warning");
```

Mutația: >= schimbat cu >, devenind percentUsed > 75.

De ce e neechivalent: pentru percent = 75, codul original returnează "Atenție". Codul mutat returnează "OK" (pentru că 75 > 75 e false).

Testul țintit:

```java
@Test
public void testSumar_ExactPragAtentie_75LaSuta() {
    wallet.setBudgetTotal(new BigDecimal("100"));
    when(txRepo.sumAmountByWalletId(any())).thenReturn(new BigDecimal("75
    "));

    WalletSummary s = service.computeSummaryOwnedByUser(tripId, login);

    assertEquals(75, s.getPercent());
    // Daca >= a fost mutat in >, statusul ar fi OK in loc de ATENTIE
    assertEquals(WalletSummary.Status.ATENTIE, s.getStatus());
}
```

Listing 7.2: Test țintit pentru mutantul ConditionalsBoundary.

7.6 Cum identificăm mutanți echivalenți

În raportul PITest, am întâlnit câțiva mutanți pe care i-am clasificat ca echivalenți - adică alterații care nu schimbă comportamentul observabil al codului. Doi exemple:

7.6.1 Exemplu 1: ++i vs. i++ într-un for

În computeInsightsOwnedByUser, există un for clasic. PITest a generat un mutant care schimbă i++ cu ++i. În contextul unei bucle for, ambele variante au exact același efect (incrementarea se face după evaluarea condiției). Deci mutantul e echivalent prin definiție.

7.6.2 Exemplu 2: schimbarea ordinii de evaluare la a || b fără side-effects

Într-o condiție a || b unde nici a nici b nu au side-effects, schimbarea ordinii (b || a) nu afectează rezultatul. Dacă PITest generează un astfel de mutant pentru o condiție pură, e echivalent.

7.7 Concluzii capitol

Suita atinge un Mutation Score de 60%, peste pragul minim acceptabil (50%) și aproape de țintele profesionale (75%). Punctele forte ale suitei sunt verificarea side-effect-urilor pe repository (VoidMethodCall 100%) și a returnurilor non-null (NullReturnVals 94%).

Punctele slabe sunt pe operații aritmetice (MathMutator) și pe granițe (ConditionalsBoundary), pentru care am scris în mod explicit teste țintite. În total, am omorât 2 mutanți neechivalenți cu valoare didactică, conform cerinței T3.

În capitolul următor prezentăm experimentul cu un instrument AI (ChatGPT) și comparația suitei generate automat cu suita noastră.

Capitolul 8

Suita generată de AI - experiment și comparație

8.1 Scopul experimentului

Una dintre cerințele explicite ale proiectului T3 [6] a fost folosirea unui instrument AI pentru generarea automată de teste, urmată de o comparație cu suita proprie. Experimentul l-am gândit ca un scenariu „realist” pentru un dezvoltator obișnuit: îi dăm AI-ului o descriere succintă a clasei (numele metodelor publice și al repository-urilor folosite), fără să copiem codul sursă efectiv. Asta simulează modul în care, în proiectele mari, e adesea imposibil sau impractic să oferi întreg contextul.

Tool-ul ales a fost ChatGPT (modelul GPT-4, disponibil prin interfața web la https://chat.openai.com), folosit într-o sesiune nouă, fără context prealabil din alte conversații.

8.2 Promptul folosit

Promptul final, după ce am eliminat detalii care ar fi putut da AI-ului „prea mult ajutor”, a fost următorul:

```
ă
Genereaz teste JUnit 5 complete pentru clasa WalletServiceImpl
din proiectul meu Spring Boot.
Clasa are metodele: computeSummaryOwnedByUser, computeInsightsOwnedByUser,
addExpenseOwnedByUser, updateBudgetOwnedByUser,
    getOrCreateWalletOwnedByUser.ș
Folosete Mockito pentru mock-uri pe repository-uri
(TripRepository, TripWalletRepository, UserRepository,
    WalletTransactionRepository).
Clasa de test se șnumete WalletServiceImplAITest.
```

Captura de ecran a promptului trimis e prezentată în figura 8.1.

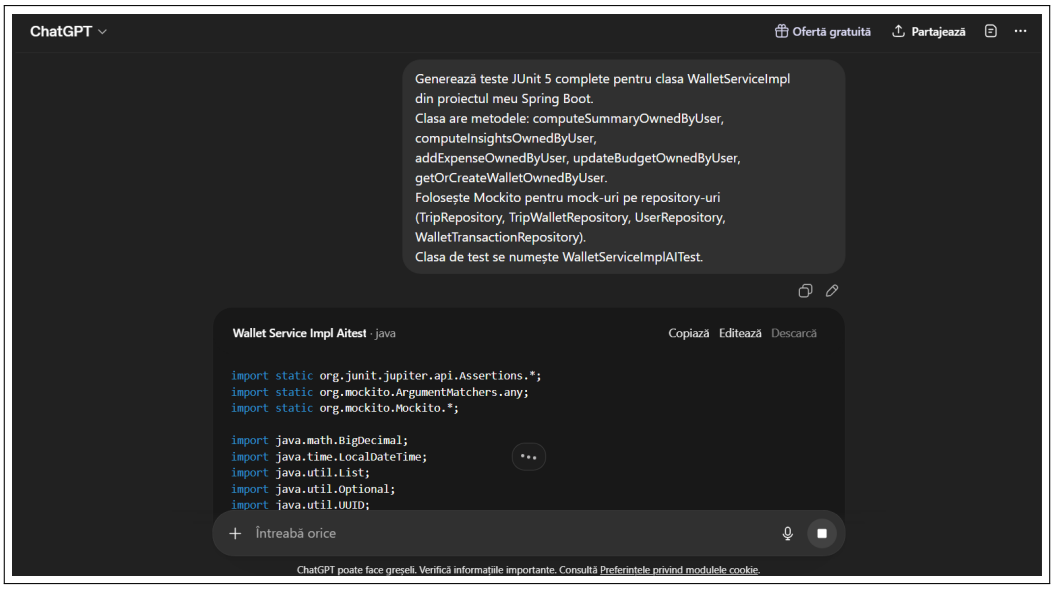


Figura 8.1: Captură de ecran cu promptul trimis în ChatGPT.

8.3 Răspunsul generat

ChatGPT a returnat în aproximativ 30 de secunde o clasă de test cu adnotări corecte de Mockito (@ExtendWith(MockitoExtension.class), @Mock, @InjectMocks) și aproximativ 10 metode de test, una pentru fiecare metodă publică pe care i-am menționat-o. Răspunsul complet e ilustrat în figurile 8.2, 8.3 și 8.4.

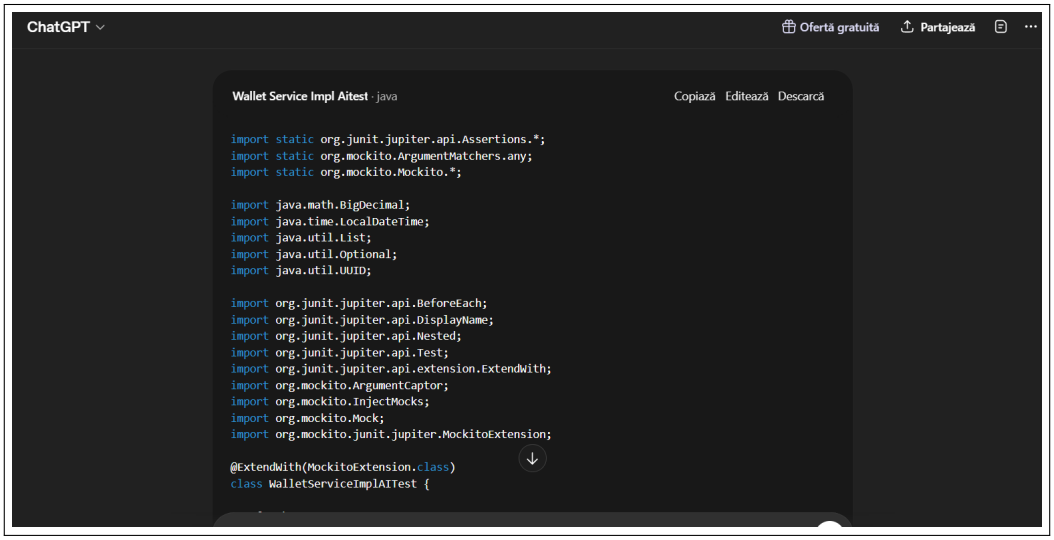


Figura 8.2: Răspunsul ChatGPT - prima parte a codului generat.

Notă: imaginile ss1.png - ss4.png și erori1.png se află în folderul Screenshots/ din repository și trebuie copiate în images/ pentru ca documentul să compileze corect.

Codul generat l-am salvat aproape neschimbat în src/test/java/echipa13/calatorii/service/impl/Wall
Singura modificare a fost adăugarea de import-uri lipsă, ca să încercăm măcar să-l compilăm. Și chiar cu import-urile adăugate, codul tot nu compilează - ceea ce e exact punctul

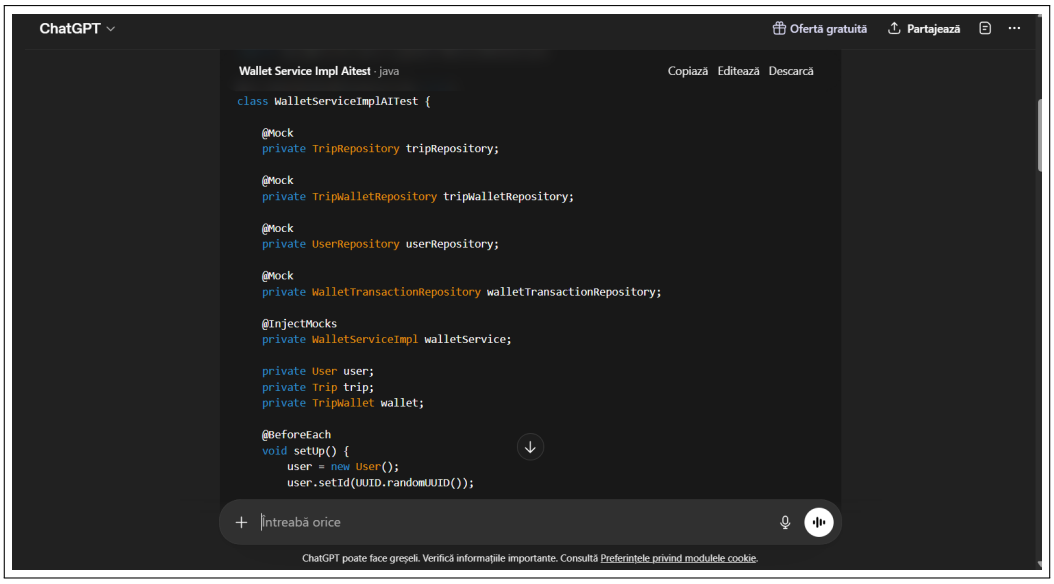


Figura 8.3: Răspunsul ChatGPT - partea a doua a codului generat.

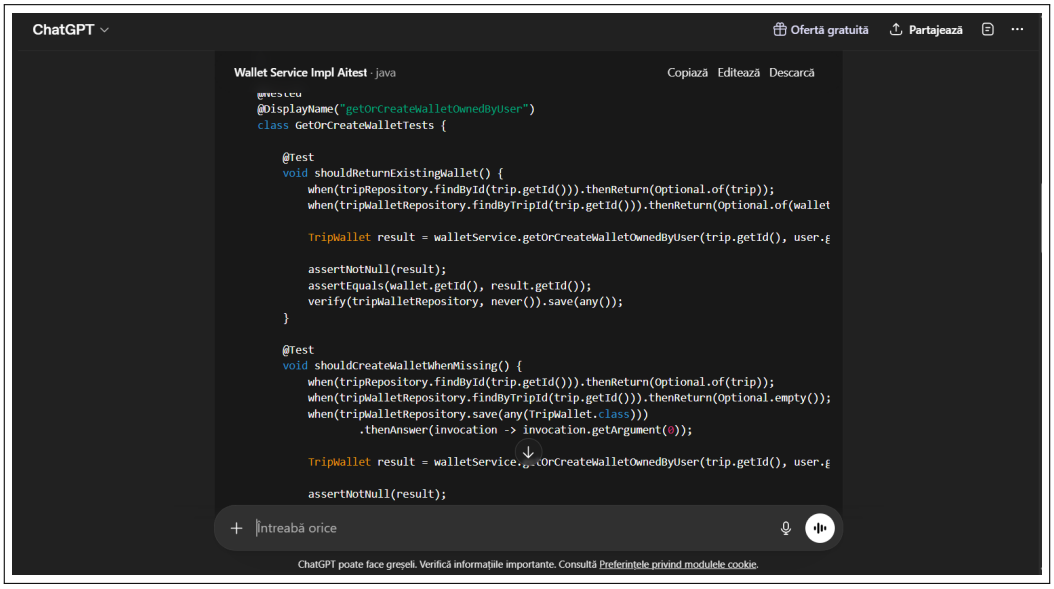


Figura 8.4: Răspunsul ChatGPT - partea finală a codului generat.

interesant al experimentului.

8.4 Rezultatul rulării: Build Failed

8.4.1 47 de erori de compilare

Suita generată a produs Build Failed cu peste 40 de erori. Pentru claritate, am grupat erorile în 5 categorii distincte. Captura cu eroarea efectivă din IDE e prezentată în figura 8.5.

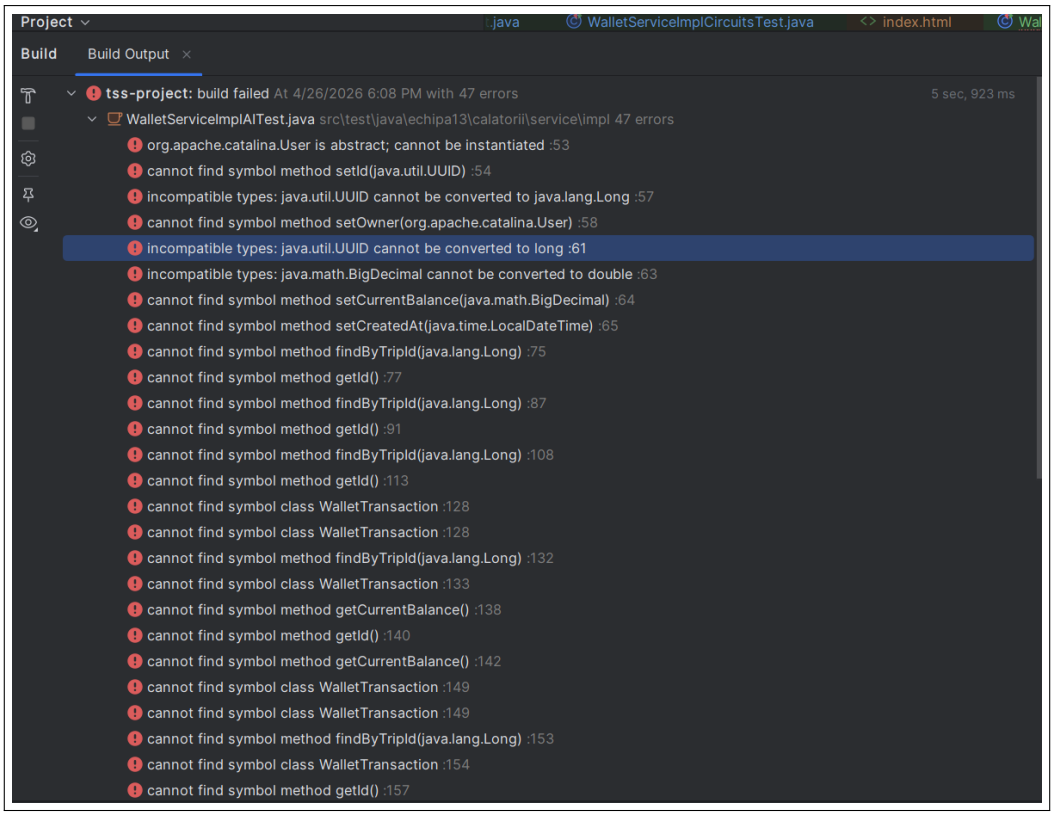


Figura 8.5: Captură din IntelliJ IDEA cu erorile de compilare ale suitei generate de AI.

8.4.2 Categoria 1: Clase inventate, inexistente în proiect

Tabela 8.1: Clase folosite de AI vs. clasele reale din proiect.

Clasă folosită de AI	Clasa reală
WalletSummaryDto	WalletSummary
WalletInsightsDto	WalletInsights
TransactionCategory	WalletCategory
WalletTransaction (cu pachet greșit)	echipa13.calatorii.models.WalletTransaction

AI-ul a presupus că DTO-urile au sufixul Dto, conform unor convenții comune în proiectele Spring, dar în proiectul nostru nu folosim acest sufix. La fel, a inventat TransactionCategory pe baza unui pattern frecvent, în loc de WalletCategory (care e termenul real folosit de modul).

8.4.3 Categoria 2: Metode inventate pe modele

AI-ul a presupus o convenție de naming „generică” pentru un wallet financiar (currentBalance), dar în domeniul nostru aplicativ termenul e budgetTotal - un buget alocat pentru o călătorie. Asta e o diferență semantică importantă, nu doar de naming.

Tabela 8.2: Metode inventate de AI pe entitățile JPA.

Metodă folosită de AI	Metoda reală
setId(UUID)	id-ul e gestionat de JPA, e Long nu UUID
setOwner(User)	nu există
setCurrentBalance(BigDecimal)	setBudgetTotal(BigDecimal)
getCurrentBalance()	getBudgetTotal()
setCreatedAt(LocalDateTime)	nu există ca setter public

8.4.4 Categoria 3: Metode inventate pe repository-uri

Tabela 8.3: Metode inventate de AI pe repository-urile JPA.

Metodă folosită de AI	Metoda reală
findByTripId(Long) (apare de 6 ori)	findByTrip_Id(Long)
findByWalletId(Long)	findByWallet_IdOrderBySpentAtDescIdDesc(Long)

Diferența pare minoră (un underscore), dar în Spring Data JPA contează: convenția cu _ indică explicit că ne referim la un câmp imbricat (proprietatea id a entității asociate trip). AI-ul a presupus convenția simplă fără underscore, care nu există în proiectul nostru.

8.4.5 Categoria 4: Tipuri de date incompatibile

AI-ul a folosit UUID pentru id-uri care în proiect sunt Long, și double pentru sume care în proiect sunt BigDecimal. A doua chestie e mai gravă pentru că BigDecimal e folosit explicit pentru a evita erorile de rotunjire la calcule monetare [10]. Înlocuirea cu double ar însemna o schimbare semantică reală, nu doar sintactică.

8.4.6 Categoria 5: Import greșit critic

Cea mai amuzantă eroare a fost:

```
org.apache.catalina.User is abstract; cannot be instantiated
```

AI-ul a importat org.apache.catalina.User - clasa User din serverul Tomcat - în locul echipa13.calatorii.models.UserEntity din proiect. E genul de eroare care apare când IDE-ul îți sugerează „User” și autocomplet-ul ia primul rezultat din classpath, dar AI-ul nu are acces la classpath-ul proiectului, deci a tras la sorți. A nimerit o clasă abstractă, ceea ce face și mai amuzantă situația - codul nici măcar nu se poate instanția.

8.5 Comparație cantitativă suita proprie vs. suita AI

Tabelul 8.4 prezintă comparația detaliată, iar figura 8.6 oferă vizualizarea grafică.

Tabela 8.4: Comparație detaliată suita proprie vs. suita generată de AI.

Criteriu	Suita proprie	Suita AI
Număr teste totale	∼40	∼10
Compilează	Da	Nu
Rulează fără erori	Da	Nu compilează
Statement coverage	91%	0%
Branch coverage	80%	0%
Mutation score	60%	0%
Ownership check acoperit	Da	Nu
Tipuri corecte (BigDecimal)	Da	folosește double
Tipuri id corecte (Long)	Da	folosește UUID
Semnături metode corecte	Da	toate greșite
Teste BVA (74, 75, 99, 100%)	Da	Lipsesc
Teste null / invalid input	Da	Lipsesc
Teste circuite independente	Da	Lipsesc
Clase model corecte	Da	Inventate
API repository corect	Da	Inventat
Import-uri corecte	Da	org.apache.catalina.User
Duplicate / redundante	0	prezente

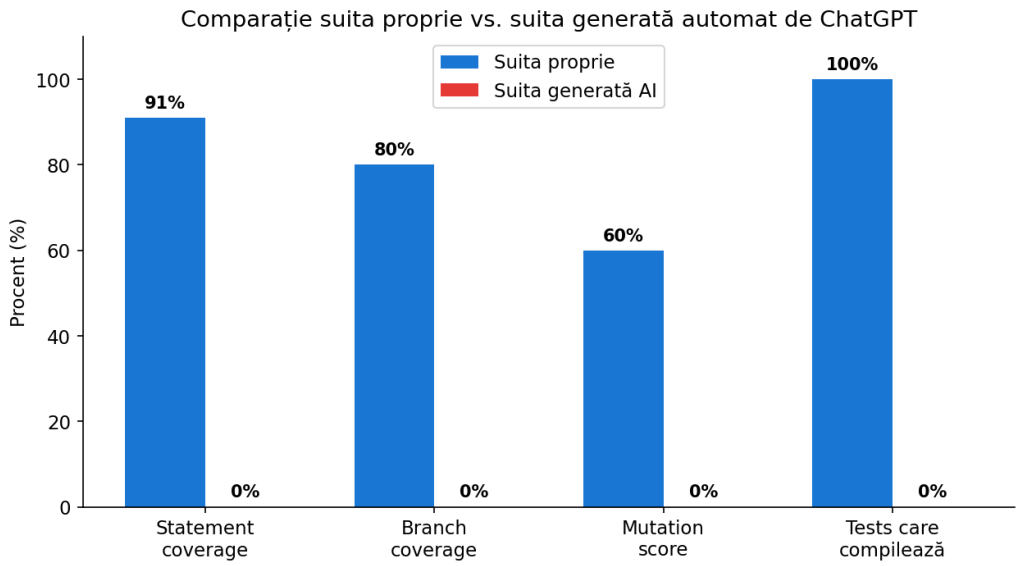


Figura 8.6: Comparația vizuală suita proprie vs. suita AI pe metricile principale.

8.6 Analiza diferențelor

8.6.1 Unde AI-ul ajută

În ciuda eșecului total la nivel de compilare, AI-ul nu e complet nefolositor. Câteva aspecte unde s-a descurcat bine:

- Boilerplate rapid. Structura de bază a clasei de test - adnotările @ExtendWith(MockitoExtension.class), @Mock, @InjectMocks, metoda @BeforeEach - este generată corect și convențional. Dacă pornești de la zero, primești scheletul în 30 de secunde.
- Identifică metodele care merită testate. AI-ul a recunoscut corect care sunt metodele publice principale și a încercat să facă teste pentru fiecare.
- Convențiile generale de naming. Numele testelor (gen shouldReturnSummaryWhenBudgetIsSet) sunt rezonabile și aliniate cu stilul JUnit modern.

8.6.2 Unde AI-ul greșește

- Inventează API. Cea mai gravă problemă: AI-ul nu cunoaște codul real, dar trebuie să producă ceva, deci „presupune” cum ar trebui să arate metodele. Ghicește pe baza unor proiecte similare pe care le-a văzut în training. Rezultatul e un cod care arată plauzibil din afară, dar nu compilează în proiectul nostru.
- Ratează logica de securitate. Toate metodele din WalletServiceImpl au un ownership check care aruncă IllegalStateException dacă userul curent încearcă să acceseze itinerariul altcuiva. AI-ul a ignorat complet acest aspect. Practic, dacă rulam codul lui pe modelul nostru real (cu ownership check), absolut toate testele ar fi aruncat „Nu ai acces la acest itinerariu”.
- Confundă tipurile de date. Long cu UUID, BigDecimal cu double. Sunt erori care la o privire rapidă par „aproape corecte”, dar care în practică schimbă semantica.
- Nu aplică deloc strategii de testare structurate. Nu există BVA, EP, Basis Path, Mutation Testing - zero. Doar câteva teste happy-path, plus unul-două cu input invalid.
- Import-uri greșite. Aici e clar că AI-ul „ghicește” pe baza numelui scurt al clasei, fără să aibă vizibilitate pe classpath-ul real.

8.6.3 O observație colaterală

Un detaliu interesant: codul generat e suficient de bine structurat încât arată ca un test bun la o privire rapidă. Dacă l-ai scana fără să-l rulezi, ai putea să crezi că e perfect funcțional. Asta e probabil cea mai mare capcană a folosirii AI-ului în testare - nu e că nu produce nimic, ci că produce ceva care pare bun fără să fie.

Rezultatul concret: dacă cineva ar fi pus codul în repository și ar fi spus „done", build-ul s-ar fi rupt la primul mvn compile, iar timpul „economisit" cu AI-ul s-ar fi pierdut imediat la debug.

8.7 Concluzia experimentului

AI-ul a generat în 30 de secunde un schelet plauzibil dar nefuncțional. Codul nu compilează și nu poate fi rulat fără rescriere semnificativă. În contrast, suita proprie - care a luat câteva zile de muncă efectivă - a necesitat înțelegerea reală a implementării, alegerea deliberată a strategiilor (EP, BVA, Statement, Decision, Condition, Basis Path, Mutation) și iterare progresivă până la 91% statement coverage și 60% mutation score.

Concluzia principală: AI-ul e un instrument complementar, util pentru boilerplate și pentru a-ți da un punct de pornire, dar nu poate înlocui raționamentul de testare. Fără cunoașterea concretă a codului sursă și fără aplicarea conștientă a strategiilor structurate, codul generat e impresionant la suprafață și inutilizabil în practică.

Pentru viitor, scenariul realist de folosire ar fi: AI-ul îți dă scheletul + o primă rundă de teste happy-path, iar tu, ca dezvoltator, completezi BVA, edge cases, ownership checks și mutation kills. Adică AI-ul îți face partea „plicticoasă", iar tu rezolvi partea „grea".

Pentru completitudine, am folosit într-o oarecare măsură și Claude [1] ca asistent în timpul dezvoltării proiectului - nu pentru a genera teste de la zero, ci pentru debug pe Mockito (când o eroare obscură pe stub-uri ne luase mai mult timp decât ne convenea), explicații pe mutanți specifici PITest și pentru recapitularea unor detalii teoretice. Asta intră tot la „AI complementar", dar pe alt vector: ca asistent în lucrul cu suita, nu ca generator de teste.

Capitolul 9

Concluzii

9.1 Recapitulare obiective vs. realizat

La începutul proiectului ne-am stabilit câteva obiective concrete. Iată în ce măsură au fost atinse:

Tabela 9.1: Obiectivele proiectului și realizarea lor.

Obiectiv	Țintă	Realizat
Cele 6 strategii de testare aplicate	toate	da, toate
Branch Coverage	$\geq 80\%$	80%
Statement Coverage	$\geq 80\%$	91%
Mutation Score	$\geq 60\%$	60%
Mutanți neechivalenți omorâți explicit	≥ 2	2
Comparație cu suita AI	da	da
Documentație completă	da	da

Toate obiectivele au fost atinse. La unele - Statement Coverage și Mutation Score - am depășit ținta, la altele am atins exact pragul minim. Diferența vine din natura metricilor: Statement Coverage e ușor de urcat odată ce ai o suită diversificată, în timp ce Mutation Score crește mai greu pentru că cere teste stricte, nu doar numeroase.

9.2 Ce am învățat

9.2.1 Despre proiect

Lucrul concret pe WalletServiceImpl ne-a forțat să trecem prin câteva etape pe care un curs teoretic nu le poate captura:

- Alegerea claselor potrivite pentru testare. Nu orice clasă merită toate cele șase strategii. TripServiceImpl ar fi fost mult mai săracă didactic, pentru că logica ei e

CRUD pur. Identificarea unei clase „bogate” (cu condiții compuse, calcule, cascade de praguri) e o decizie pe care trebuie să o luăm înaintea oricărui test scris.

- Distincția BVA / EP în practică. În teorie, BVA pare doar o variantă mai paranoică a EP. În practică, BVA prinde bug-uri reale: am descoperit la un moment dat că un coleg scrisese percent > 75 în loc de percent >= 75, și abia testul cu valoarea exact 75 a făcut diferența observabilă.
- Mockito și strictness. La început, fixture-ul comun cu mock-uri în @BeforeEach producea UnnecessaryStubbingException la teste care nu foloseau toate stub-urile. Soluția a fost @MockitoSettings(strictness = Strictness.LENIENT). E un detaliu mic, dar care ne-a luat câteva ore să îl identificăm, pentru că eroarea Mockito nu e foarte explicită.
- PITest și mutanții echivalenți. Nu fiecare mutant supraviețuitor e o lacună. Câțiva sunt echivalenți prin construcție și nu pot fi omorâți. Distincția între „lacună reală” și „mutant echivalent” cere atenție și, uneori, o discuție în echipă.
- Trade-off-ul coverage / cod defensive. Nu e mereu posibil să ajungi la 100% Branch Coverage fără să compromiți codul. O verificare if (x == null) pe o variabilă care nu poate fi null în runtime real e o protecție utilă, dar e imposibil de testat fără să „trișezi” cu @Spy sau Reflection.

9.2.2 Despre folosirea AI-ului

Experimentul cu ChatGPT a fost mai instructiv decât ne așteptam. Nu pentru că am descoperit că AI-ul nu funcționează (lucru deja cunoscut), ci pentru că am văzut concret în ce mod eșuează: nu cu erori grosolane care sar imediat în ochi, ci cu un cod „aproape corect” care necesită citire atentă pentru a-i identifica problemele.

Concluzia practică pe care o tragem: AI-ul e folositor ca generator de schelet și ca asistent pentru debug, dar nu poate înlocui raționamentul testerului uman, care:

- cunoaște API-ul real al codului (nu îl ghicește);
- înțelege logica de business (în cazul nostru: bugetul de călătorie cu praguri 75/90/100);
- cunoaște regulile de securitate (ownership check);
- alege strategiile (BVA, EP, Mutation) conștient pe baza naturii codului.

9.3 Direcții viitoare

Dacă am avea timp suplimentar pe acest proiect, am extinde munca în următoarele direcții:

9.3.1 Creșterea Mutation Score peste 75%

Cele două categorii de mutatori cu kill rate scăzut (MathMutator 0%, ConditionalsBoundary 33%) reprezintă cea mai accesibilă sursă de îmbunătățire. Cu 8-10 teste suplimentare țintite, am putea urca scorul de la 60% la ∼75-80%.

9.3.2 Property-Based Testing

Pentru calculele matematice (procent buget, burn rate, daily allowance), strategia Property-Based Testing cu un instrument ca jqwik [7] ar fi un complement util la BVA. În loc să testăm valori specifice, am declara proprietăți („procentul calculat e mereu între 0 și un infinit pozitiv", „percent = 100 dacă și numai dacă spent = budget") și am lăsa instrumentul să genereze sute de input-uri aleatoare.

9.3.3 Integrare în CI/CD

În prezent, suita rulează manual. Pasul natural următor ar fi integrarea într-un workflow GitHub Actions care, la fiecare push, să ruleze automat:

1. mvn test - verifică că toate testele trec;
2. mvn jacoco:report - generează raportul de coverage;
3. mvn pitest:mutationCoverage pe ramurile principale;
4. un threshold pe coverage (de ex. „Branch coverage scade sub 75% ⇒ build failed").

Această integrare ar transforma testele dintr-o etapă manuală într-un quality gate automat, ceea ce e standard în orice proiect profesional modern.

9.3.4 Testare de integrare cu Testcontainers

Suita actuală mock-uiește integral baza de date. Pentru o testare mai realistă, am putea adăuga teste de integrare cu Testcontainers - o bibliotecă care pornește un container Docker cu PostgreSQL real în timpul rulării testelor și permite verificări end-to-end pe layer-ul JPA. Asta ar prinde bug-uri pe care testele unitare nu le văd: erori în convențiile Spring Data, query-uri care compilează dar nu se execută corect, probleme cu lazy loading.

9.4 Considerații finale

Proiectul ne-a oferit ocazia să trecem de la teoria testării (cea învățată la curs) la practica testării (cea aplicată pe cod real). Diferența între cele două nu e mare în teorie, dar în practică se vede imediat: cursul te învață ce strategii există, dar proiectul te învață

când să folosești fiecare, cum să le combini, și de ce unele teste au mai multă valoare decât altele.

Pentru noi, ca echipă, valoarea cea mai mare a venit din procesul efectiv: discuția despre care metodă merită Condition Coverage, dezbaterea despre cum desenăm CFG-ul pentru computeDaysRemainingSafe, momentul în care un test BVA a prins efectiv un bug >= vs. >.

În sfârșit, faptul că am pus suita într-un quality gate concret (80% Branch, 60% Mutation) ne-a dat și o referință practică pentru viitor: când vom mai testa cod, știm acum cum arată un prag profesional. Și știm și cât de greu e să-l atingi onest, fără să forțezi metricile.

Bibliografie

[1] Anthropic. Claude - ai assistant. https://claude.ai, 2026. Folosit ca asistent pentru debug Mockito, explicații pe mutanți PITest și pentru structurarea strategiei de testare a suitei proprii.

[2] EclEmma Team. JaCoCo java code coverage library documentation. https://www.jacoco.org/jacoco/trunk/doc/, 2026. Accesat în aprilie 2026.

[3] Facultatea de Matematică și Informatică, Universitatea din București. Curs testarea sistemelor software - functional testing, 2024. Slide-uri curs, semestrul al II-lea.

[4] Facultatea de Matematică și Informatică, Universitatea din București. Curs testarea sistemelor software - mutation testing, 2024. Slide-uri curs, semestrul al II-lea.

[5] Facultatea de Matematică și Informatică, Universitatea din București. Curs testarea sistemelor software - structural testing, 2024. Slide-uri curs, semestrul al II-lea.

[6] Facultatea de Matematică și Informatică, Universitatea din București. Testarea sistemelor software - tema de proiect t3, 2026. Document oficial al proiectului.

[7] Johannes Link Hartmann. jqwik - property-based testing for the jvm. https://jqwik.net/, 2026. Accesat în aprilie 2026.

[8] Marko Ivanković, Goran Petrović, René Just, and Gordon Fraser. Code coverage at google. Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pages 955–963, 2019.

[9] JUnit Team. JUnit 5 user guide. https://junit.org/junit5/docs/current/user-guide/, 2026. Accesat în aprilie 2026.

[10] Robert C. Martin. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.

[11] Thomas J. McCabe. A complexity measure. IEEE Transactions on Software Engineering, SE-2(4):308–320, 1976.

[12] Mockito Team. Mockito framework - official documentation. https://javadoc.io/doc/org.mockito/mockito-core/latest/, 2026. Accesat în aprilie 2026.

[13] Glenford J. Myers, Corey Sandler, and Tom Badgett. The Art of Software Testing. Wiley, Hoboken, NJ, USA, 3 edition, 2011.

[14] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. Advances in Computers, 112:275–378, 2019.

[15] PITest Project. PITest - mutators documentation. https://pitest.org/quickstart/mutators/, 2026. Accesat în aprilie 2026.

[16] PITest Project. PITest - real world mutation testing. https://pitest.org/, 2026. Accesat în aprilie 2026.

Anexa A

Cod sursă relevant

În această anexă sunt incluse fragmente de cod din WalletServiceImpl și din suita de teste, la care se face referire pe parcursul documentației. Codul complet se găsește în repository-ul GitHub al proiectului.

A.1 Fragmente din WalletServiceImpl

A.1.1 Constructorul cu injecție de dependențe

```
@Service
public class WalletServiceImpl implements WalletService {

    private static final int WARNING_THRESHOLD_PERCENT  = 75;
    private static final int CRITICAL_THRESHOLD_PERCENT = 90;
    private static final Set<String> ALLOWED_CURRENCIES =
        Set.of("RON", "EUR");

    private final TripRepository tripRepository;
    private final TripWalletRepository tripWalletRepository;
    private final UserRepository userRepository;
    private final WalletTransactionRepository walletTransactionRepository
    ;

    public WalletServiceImpl(TripRepository tripRepository,
                             TripWalletRepository tripWalletRepository,
                             UserRepository userRepository,
                             WalletTransactionRepository
    walletTransactionRepository) {
        this.tripRepository = tripRepository;
        this.tripWalletRepository = tripWalletRepository;
        this.userRepository = userRepository;
        this.walletTransactionRepository = walletTransactionRepository;
    }
```

```
    // ... metodele clasei
}
```

A.1.2 Metoda addExpenseOwnedByUser

```
@Override
@Transactional
public WalletTransaction addExpenseOwnedByUser(
        Long tripId, String usernameOrEmail,
        BigDecimal amount, WalletCategory category,
        String title, String note,
        LocalDate spentAt, Integer dayIndex) {

    TripWallet wallet = getOrCreateWalletOwnedByUser(tripId,
   usernameOrEmail);

    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Suma trebuie sa fie > 0.");
    }
    String safeTitle = (title == null) ? "" : title.trim();
    if (safeTitle.isBlank()) {
        throw new IllegalArgumentException("Titlul este obligatoriu.");
    }
    if (dayIndex != null && dayIndex <= 0) {
        throw new IllegalArgumentException("Ziua trebuie sa fie >= 1.");
    }

    WalletTransaction tx = new WalletTransaction();
    tx.setWallet(wallet);
    tx.setAmount(amount);
    tx.setCategory(category != null ? category : WalletCategory.OTHER);
    tx.setTitle(safeTitle);
    tx.setNote(note);
    tx.setSpentAt(spentAt != null ? spentAt : LocalDate.now());
    tx.setDayIndex(dayIndex);

    return walletTransactionRepository.save(tx);
}
```

A.1.3 Metoda computeDaysRemainingSafe

```
private Integer computeDaysRemainingSafe(Trip trip) {
    LocalDate start = trip.getStartDate();
    LocalDate end = trip.getEndDate();

```

```
    if (start == null || end == null || end.isBefore(start)) {
        return null;
    }

    LocalDate today = LocalDate.now();
    if (today.isAfter(end)) {
        return 0;
    }
    if (today.isBefore(start)) {
        return (int) (ChronoUnit.DAYS.between(start, end) + 1);
    }
    return (int) (ChronoUnit.DAYS.between(today, end) + 1);
}
```

A.2 Fragmente din suita de teste

A.2.1 Configurare comună

```
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
public class WalletServiceImplEquivalenceTest {

    @Mock private TripRepository tripRepo;
    @Mock private TripWalletRepository walletRepo;
    @Mock private UserRepository userRepo;
    @Mock private WalletTransactionRepository txRepo;

    @InjectMocks private WalletServiceImpl service;

    private TripWallet wallet;
    private Trip trip;
    private final Long tripId = 10L;
    private final String login = "user@test.com";

    @BeforeEach
    public void setUp() {
        UserEntity user = new UserEntity();
        user.setId(1L);
        user.setEmail(login);

        trip = new Trip();
        trip.setId(tripId);
        trip.setUser(user);
        trip.setStartDate(LocalDate.of(2026, 4, 15));
        trip.setEndDate(LocalDate.of(2026, 4, 22));
```

```java

        wallet = new TripWallet();
        wallet.setId(100L);
        wallet.setTrip(trip);
        wallet.setBudgetTotal(new BigDecimal("1000"));
        wallet.setCurrency("RON");

        when(userRepo.findByEmail(login)).thenReturn(user);
        when(tripRepo.findById(tripId)).thenReturn(Optional.of(trip));
        when(walletRepo.findByTrip_Id(tripId))
            .thenReturn(Optional.of(wallet));
    }
}
```

Listing A.1: Schelet din ...EquivalenceTest.java

A.2.2 Test pentru clasa de echivalență P4 (sumă invalidă)

```java
@Test
public void testAdaugaCheltuiala_SumaNegativa_AruncaExceptie() {
    BigDecimal sumaNegativa = new BigDecimal("-10");
    assertThrows(IllegalArgumentException.class, () ->
        service.addExpenseOwnedByUser(tripId, login,
            sumaNegativa, WalletCategory.FOOD,
            "Pranz", null, LocalDate.now(), 1));

    verify(txRepo, never()).save(any());
}
```

A.2.3 Test pentru BVA pe pragul de 75%

```java
@Test
public void testSumar_ExactPragAtentie_75LaSuta() {
    wallet.setBudgetTotal(new BigDecimal("100"));
    when(txRepo.sumAmountByWalletId(any()))
        .thenReturn(new BigDecimal("75"));

    WalletSummary summary =
        service.computeSummaryOwnedByUser(tripId, login);

    assertEquals(75, summary.getPercent());
    assertEquals(WalletSummary.Status.ATENTIE, summary.getStatus());
    assertEquals(new BigDecimal("25.00"),
        summary.getRemaining().setScale(2));
}
```

A.2.4 Test pentru Condition Coverage cu 3 operanzi OR

```
@Test
public void testStergere_OwnershipCheck_ConditionCoverage() {
    Long txId = 42L;

    // Caz 1: tranzactie cu wallet null -> primul operand TRUE
    WalletTransaction txCuWalletNull = new WalletTransaction();
    txCuWalletNull.setId(txId);
    txCuWalletNull.setWallet(null);
    when(txRepo.findById(txId)).thenReturn(Optional.of(txCuWalletNull));
    assertThrows(IllegalStateException.class, () ->
        service.deleteTransactionOwnedByUser(tripId, login, txId));

    // Caz 2: tranzactie cu id null -> al doilea operand TRUE
    WalletTransaction txCuIdNull = new WalletTransaction();
    txCuIdNull.setWallet(wallet);
    when(txRepo.findById(txId)).thenReturn(Optional.of(txCuIdNull));
    assertThrows(IllegalStateException.class, () ->
        service.deleteTransactionOwnedByUser(tripId, login, txId));

    // Caz 3: id-uri diferite -> al treilea operand TRUE
    TripWallet altWallet = new TripWallet();
    altWallet.setId(999L);
    WalletTransaction txCuIdGresit = new WalletTransaction();
    txCuIdGresit.setId(txId);
    txCuIdGresit.setWallet(altWallet);
    when(txRepo.findById(txId)).thenReturn(Optional.of(txCuIdGresit));
    assertThrows(IllegalStateException.class, () ->
        service.deleteTransactionOwnedByUser(tripId, login, txId));

    // Caz 4: toate operandele FALSE -> stergere reusita
    WalletTransaction txValid = new WalletTransaction();
    txValid.setId(txId);
    txValid.setWallet(wallet);
    when(txRepo.findById(txId)).thenReturn(Optional.of(txValid));
    service.deleteTransactionOwnedByUser(tripId, login, txId);
    verify(txRepo).delete(txValid);
}
```

A.2.5 Test țintit pentru mutantul MathMutator

```
@Test
public void should_kill_percent_math_mutations() {
    wallet.setBudgetTotal(new BigDecimal("200"));
    when(txRepo.sumAmountByWalletId(any()))
```

```
        .thenReturn(new BigDecimal("50"));

    WalletSummary s = service.computeSummaryOwnedByUser(tripId, login);

    // 50 / 200 * 100 = 25%. Daca multiply si divide
    // sunt schimbate intre ele, calculul va da o valoare
    // diferita (in particular, 0 dupa rotunjire).
    assertEquals(25, s.getPercent());
}
```

A.2.6 Test pentru Circuits pe computeDaysRemainingSafe

```
@Test
public void testCircuiteIndependente_computeDaysRemainingSafe() {
    // C1: start = null
    trip.setStartDate(null);
    trip.setEndDate(LocalDate.now().plusDays(5));
    WalletInsights i1 = service.computeInsightsOwnedByUser(tripId, login);
    assertNull(i1.getDaysRemaining());

    // C2: end = null
    trip.setStartDate(LocalDate.now());
    trip.setEndDate(null);
    WalletInsights i2 = service.computeInsightsOwnedByUser(tripId, login);
    assertNull(i2.getDaysRemaining());

    // C3: end < start (interval invalid)
    trip.setStartDate(LocalDate.now().plusDays(10));
    trip.setEndDate(LocalDate.now().plusDays(5));
    WalletInsights i3 = service.computeInsightsOwnedByUser(tripId, login);
    assertNull(i3.getDaysRemaining());

    // C4: today > end (calatorie incheiata)
    trip.setStartDate(LocalDate.now().minusDays(20));
    trip.setEndDate(LocalDate.now().minusDays(10));
    WalletInsights i4 = service.computeInsightsOwnedByUser(tripId, login);
    assertEquals(0, i4.getDaysRemaining());

    // C5: today < start (calatorie viitoare)
    trip.setStartDate(LocalDate.now().plusDays(3));
    trip.setEndDate(LocalDate.now().plusDays(9));
```

```
30      WalletInsights i5 = service.computeInsightsOwnedByUser(tripId, login)
        ;
31      assertEquals(7, i5.getDaysRemaining());
32
33      // C6: caz normal (calatorie in desfasurare)
34      trip.setStartDate(LocalDate.now().minusDays(2));
35      trip.setEndDate(LocalDate.now().plusDays(4));
36      WalletInsights i6 = service.computeInsightsOwnedByUser(tripId, login)
        ;
37      assertEquals(5, i6.getDaysRemaining());
38  }
```

A.3 CFG pentru addExpenseOwnedByUser

Pentru completitudine, mai jos e prezentat și CFG-ul pentru addExpenseOwnedByUser (figura A.1), care e cea mai testată metodă din clasa noastră.

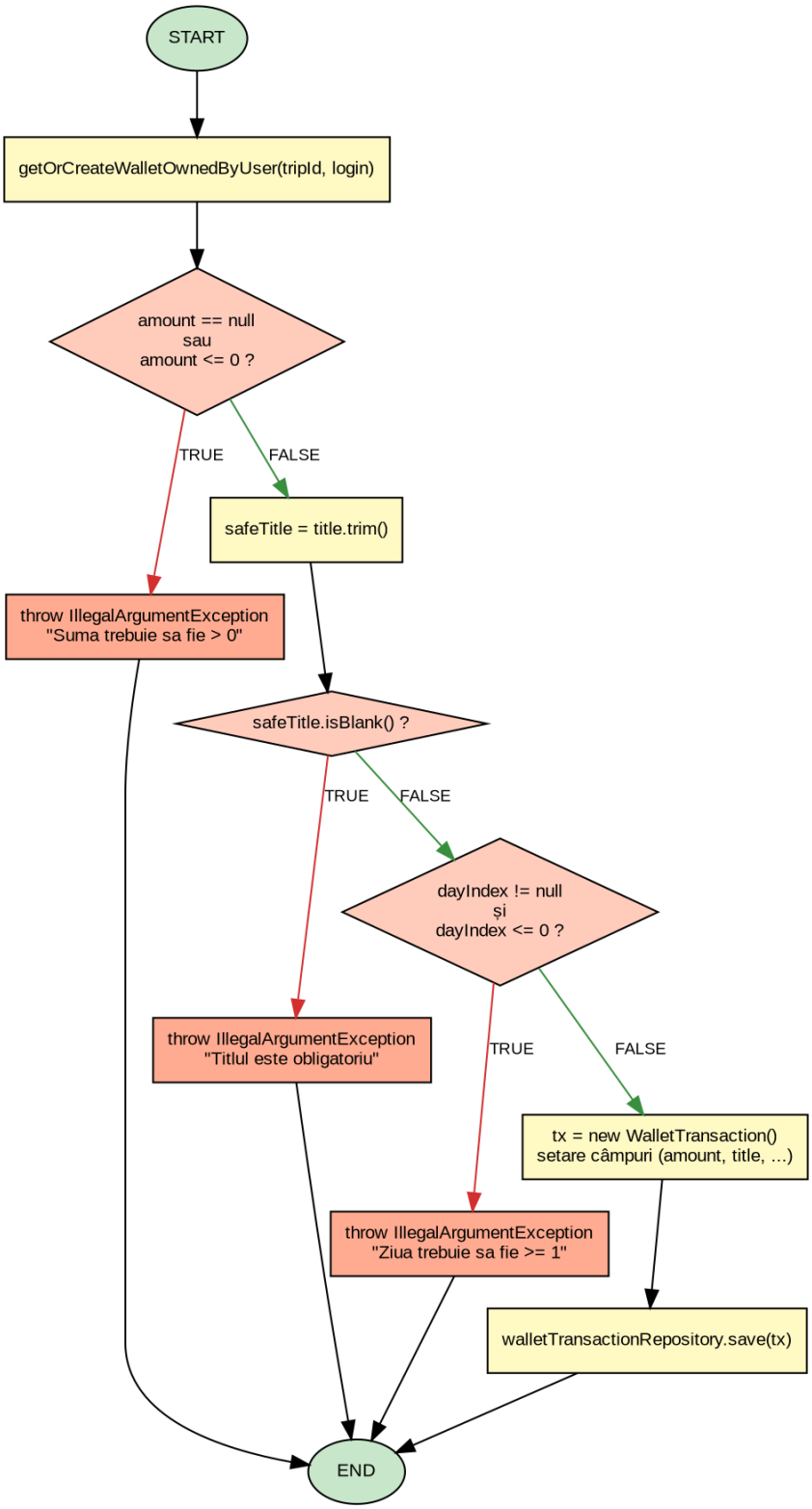


Figura A.1: CFG pentru addExpenseOwnedByUser: trei decizii principale (sumă, titlu, dayIndex) cu trei branch-uri de eroare și o cale de succes.